TRƯỜNG ĐẠI HỌC CÔNG NGHỆ SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

Giáo trình thực hành:

NHẬP MÔN CẤU TRÚC DỮ LIỆU

(Lưu hành nội bộ)

TP HỒ CHÍ MINH, 2024

MỤC LỤC

1 DANH SÁCH ĐẶC 1
1.1 MỤC TIÊU: 1
1.2 TÓM TẮT LÝ THUYẾT: 1
1.3 NỘI DUNG THỰC HÀNH: 6
1.3.1 Câu 1: 6
1.3.2 Câu 2: 7
1.3.3 Câu 3: 9
1.3.4 Câu 4: 9
1.3.5 Câu 5: 9
1.4 BÀI TẬP VỀ NHÀ 10
1.4.1 Câu 1: 10
1.4.2 Câu 2: 10
1.4.3 Câu 3: 10
1.4.4 Câu 4: 10
2 GIẢI THUẬT TÌM KIẾM – TRÊN SỐ NGUYÊN & TRÊN ĐỐI TƯỢNG 11
2.1 MỤC TIÊU: 11
2.2 TÓM TẮT LÝ THUYẾT: 11
2.3 NỘI DUNG THỰC HÀNH: 12
2.3.1 Tìm kiếm tuần tự trên dãy số 12
2.3.2 Gọi hàm 14
2.3.3 Câu 2: Tìm kiếm nhị phân trên dãy số 15
2.3.4 Câu 3: Tìm tuần tự trên đối tượng 16
2.3.5 Câu 3.2: 17
2.3.6 Câu 3.3: 19
2.3.7 Câu 4: Tìm nhị phân trên đối tượng 20
2.4 BÀI TẬP VỀ NHÀ: 20
2.4.1 Câu 1: 20
2.4.2 Câu 2: 20
2.4.3 Câu 3: 20

3 DANH SÁCH LIÊN KẾT ĐƠN 21
3.1 MỤC TIÊU 21
3.2 TÓM TẮT LÝ THUYẾT 21
3.3 NỘI DUNG THỰC HÀNH 24
3.3.1 Câu 1: 24
3.3.2 Câu 2: 26
3.3.3 Câu 3: 26
3.3.4 Câu 4: 26
3.3.5 Câu 5: 27
3.3.6 Câu 6: 28
3.3.7 Câu 7: 28
3.4 BÀI TẬP VỀ NHÀ 28
3.4.1 Câu 1: 28
3.4.2 Câu 2: 28
4 DANH SÁCH LIÊN KẾT ĐƠN – TRÊN ĐỐI TƯỢNG 29
4.1 MỤC TIÊU: 29
4.2 TÓM TẮT LÝ THUYẾT 29
4.3 NỘI DUNG THỰC HÀNH 30
4.3.1 Câu 1: 30
4.3.2 Câu 2: 33
4.3.3 Câu 3: 33
4.3.4 Câu 4: 34
4.3.5 Câu 5: 35
4.4 BÀI TẬP VỀ NHÀ 36
4.4.1 Câu 1: 36
4.4.2 Câu 2: 36
4.4.3 Câu 3. 37
5 DANH SÁCH LIÊN KẾT KÉP, LIÊN KẾT VÒNG 38
5.1 MỤC TIÊU 38
5.2 TÓM TẮT LÝ THUYẾT 38

5.3 NỘI DUNG THỰC HÀNH 42
5.3.1 Danh sách liên kết kép 42
5.3.2 Danh sách liên kết vòng 49
5.3.3 Trên đối tượng 52
5.4 BÀI TẬP VỀ NHÀ 55
6 DANH SÁCH ĐẶC BIỆT (STACK & QUEUE) 56
6.1 PHẦN 1: STACK (NGĂN XẾP) 56
6.1.1 TÓM TẮT LÝ THUYẾT 56
6.1.2 NỘI DUNG THỰC HÀNH 57
6.2 PHẦN 2: QUEUE (HÀNG ĐỢI) 61
6.2.1 MỤC TIÊU: 61
6.2.2 TÓM TẮT LÝ THUYẾT 61
6.2.3 NỘI DUNG THỰC HÀNH 63
7 CÂY NHỊ PHÂN - CÂY NHỊ PHÂN TÌM KIẾM 66
7.1 MỤC TIÊU: 66
7.2 TÓM TẮT LÝ THUYẾT: 66
7.3 NỘI DUNG THỰC HÀNH 68
7.3.1 Câu 1: 68
7.3.2 Câu 2: 72
7.3.3 Câu 3: 72
7.3.4 Câu 4: 72
7.3.5 Câu 5: 72
8 Cây nhị phân tìm kiếm 73
9 THUẬT TOÁN SẮP XẾP 74
9.1 MỤC TIÊU: 74
9.2 TÓM TẮT LÝ THUYẾT 74
9.3 NỘI DUNG THỰC HÀNH 79
9.3.1 Câu 1: 79
9.3.2 Câu 2: 80
9.3.3 Câu 3: 81

9.3.4 Câu 4: 82
9.4 BÀI TẬP VỀ NHÀ 82
9.4.1 Câu 1: 82
9.4.2 Câu 2: 82
10 THI CUỐI KỲ 83

1 DANH SÁCH ĐẶC

1.1 MỤC TIÊU:

Trong bài thực hành này, sinh viên sẽ:

– Ôn tập lại cách khai báo và truy cập danh sách đặc (mảng một chiều).

– Ôn tập lại các tác vụ trên danh sách đặc: cài đặt, khởi tạo, duyệt, thêm phần tử, loại bỏ phần tử, thay thế giá trị, ghép danh sách.

– Ôn tập lại cách gỡ lỗi chương trình (debug)

1.2 TÓM TẮT LÝ THUYẾT:

<table>
<tr><td colspan="2">1. Ôn tập mảng</td></tr>
<tr><td><u>Khai báo:</u>
<kiểu dữ liệu> <tên mảng><[số phần tử mảng]> Lưu ý: Các phần tử của mảng sẽ được tính từ 0. Vì vậy, phần tử đầu tiên trong mảng có chỉ số là 0, phần tử cuối cùng có chỉ số là <số phần tử mảng - 1>.</td><td>int a[5];
// khai báo mảng a có tối đa 5 phần tử, mỗi phần tử có kiểu int.
// phần tử đầu tiên có chỉ số là 0, phần tử cuối cùng có chỉ số là 4.</td></tr>
<tr><td><u>Truy xuất:</u> các phần tử mảng theo cú pháp: <tên mảng><[chỉ số phần tử]></td><td>Trong khai báo mảng a ở ví dụ trên:
a[0] // truy xuất giá trị phần tử đầu tiên
a[1] // truy xuất giá trị phần tử thứ hai.
a[4] // truy xuất giá trị phần tử cuối cùng</td></tr>
<tr><td><u>hập giá trị:</u> cho từng phần tử mảng</td><td>cin>>a[0]; //nhập giá trị cho phần tử đầu.
cin>>a[4]; //nhập giá trị cho phần tử cuối cùng trong mảng a
Ngoài ra ta có thể thực hiện gán cho từng giá trị như sau:
a [0] = 5 ; a [1] = -10 ;</td></tr>
</table>

Ghi chú: Trong môn học này, chúng ta sẽ thống nhất trong một project sẽ tiến hành cài đặt theo 3 file: gồm có: thuvien_lab_n, caidat_lab_n, chuongtrinh_lab_n.

Lưu ý: n ở đây là thứ tự các bài lab

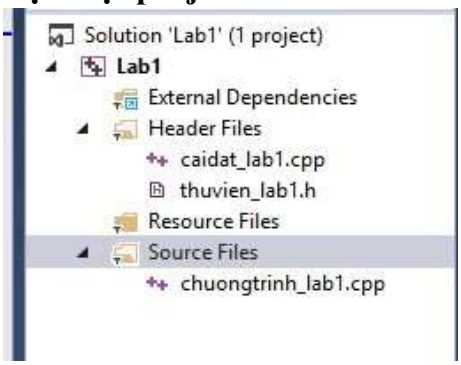
- **`thuvien_lab_n:`** Chứa các thư viện cần khai báo của chương trình

- **`caidat_lab_n:`** Chứa các hàm cài đặt của chương trình

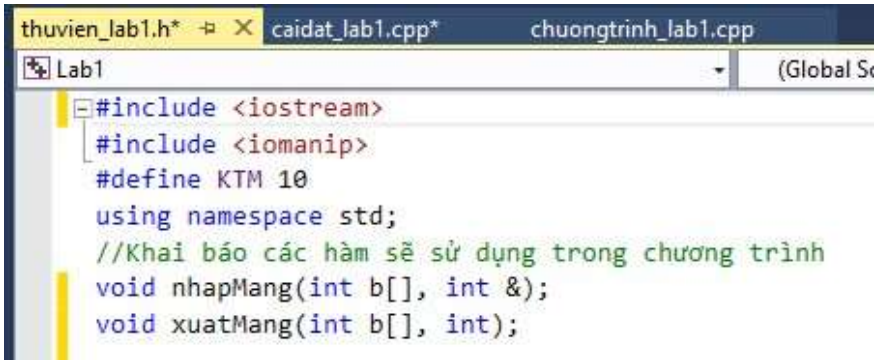
- **`chuongtrinh_lab_n:`** Chứa void main() để chạy chương trình

- Ví dụ về cách tạo 3 file trong một chương trình

Tạo một project ban đầu:



`thuvien_lab1.h`



`caidat_lab1.cpp`

huvien_lab1.h* | caidat_lab1.cpp* | chuongtrinh_lab1.cpp

Lab1 | (Global S

```
#include"thuvien_lab1.h"
//Định nghĩa các hàm sẽ xây dựng
void nhapMang(int b[], int &m){

}
void xuatMang(int b[], int){

}
```

chuongtrinh_lab1.cpp

thuvien_lab1.h* | caidat_lab1.cpp* | chuongtrinh_lab1.cpp*

Lab1 | (Global Scope)

```
#include"thuvien_lab1.h"
void main(){
    //Khai bao bien va thuc hien goi cac ham se thuc thi
    int a[KTM], n, x;
    nhapMang(a, n);
    xuatMang(a, n);
}
```

2. Ôn tập khai báo cấu trúc và các tác vụ trên dãy số: cài đặt, khởi tạo, duyệt, thêm phần tử, loại bỏ phần tử, thay thế giá trị, ghép dãy số

Biểu diễn dãy số:

Giả sử chúng ta quy ước chiều dài tối đa của dãy số là 50, khi đó cấu trúc dữ liệu để biểu diễn dãy số như ví dụ bên:

```
const int MAX=50;
```

Hoặc:

```
#define MAX 50
```

Cài đặt dãy số:
 	
Khai báo tổng quát:
 	
`struct <ten_cau_truc>{ int n;//so phan tu trong ds`
 	
`<kiểu dữ liệu> <ten_mang><[số phần tử]>;`
 	
`};`
	`const int MAX=50;`
	
	`struct DaySo {`
	
	`int ds[MAX];`
	
	`int n;`
	
	`};`

Khởi tạo dãy số:
 	
Đặt số phần tử trong danh sách bằng 0
	`void khoiTao (DaySo &day){`
	
	`day.n=0;`
	
	`}`

Kiểm tra danh sách có rỗng hay không:
 	
// Nếu số phần tử bằng không trả về true; ngược lại trả về false
	`bool kiemTraRong (DaySo day){`
	
	`return (day.n==0?true:false);`
	
	`}`

Kiểm tra danh sách có đầy hay không:
 	
// Nếu số lượng phần tử có trong danh sách bằng MAX trả về true; ngược lại trả về false
	`bool kiemTraDay(DaySo day){`
	
	`return (day.n==MAX?true:false);`
	
	`}`

Xác định số phần tử của danh sách:
 	
//Trả về số phần tử của danh sách
	`int laySoPhanTu (DaySo day){`
	
	`return day.n;`
	
	`}`

Truy xuất giá trị của phần tử tại vị trí trong dãy số:	`int truyXuatTaiViTri(DaySo day,int viTri){`
	`if(viTri <0 || viTri >layKichThuoc(day)-1)`
	`cout <<"Vi tri "<<viTri<<" khong hop le";`
	`else`

// Sinh viên tham khảo ví dụ	`if(kiemTraRong(day))`
	`cout<<"DS bi rong";`
	`else`
	`return day.ds[viTri];`
	`}`

Thêm một phần tử vào dãy số tại vị trí cho trước
// Sinh viên tham khảo ví dụ
	`void chenViTri(DaySo &day, int viTri, int a){`
	`if (viTri <0 || viTri >laySoPhanTu(day))`
	`cout << "\n Vi tri " << viTri << " khong hop le";`
	`else`
	`if (kiemTraRong(day) == true){`
	`if (pos == 0){`
	`day.ds[0] = a;`
	`day.n++;`
	`}`
	`else`
	`cout << "\n Vi tri khong hop le";`
	`}`
	`else if (kiemTraDay(day) == true)`
	`cout << "\n Danh sach day";`
	`else{`
	`for (int i = laySoPhanTu(day) - 1; i >= viTri; i--)`
	`day.ds[i + 1] = day.ds[i];`
	`day.ds[viTri] = a;`
	`day.n++;`
	`}`
	`}`

Xóa một phần tử ra khỏi dãy số tại vị trí:
// Sinh viên tham khảo ví dụ
	`void xoaViTri(DaySo &day, int viTri){`
	`int i;`
	`if(viTri <0 || viTri > laySoPhanTu(day)-1)`
	`cout <<"Vi tri"<<viTri<<"khong hop le !";`
	`else`
	`if(kiemTraRong(day)) cout<<"DS bi rong";`
	`else {`
	`for(i=viTri+1;i<=laySoPhanTu(day);i++)`
	`day.ds[i-1]= day.ds[i];`
	`day.n--;`
	`}`
	`}`

Thay thế một phần tử trong dãy số bởi	`void thayTheViTri (DaySo &day, int viTri, int a){`

một phần tử khác tại vị trí:
// Sinh viên tham khảo ví dụ
	`if (viTri<0||viTri>=laySoPhanTu(day))`
	`cout<<"Vi tri "<<viTri<<"khong tim thay !";`
	`else if (kiemTraRong(day)==true)`
	`cout<<"Danh sach bi rong " ;`
	`else day.ds[viTri]=a ;`
	`}`

3. Cách gỡ lỗi (debug)	
Đặt điểm dừng	Nhấn F9 trên dòng muốn dừng hoặc nhấn vào biên trái của cửa sổ Visual Studio
Chạy từng dòng	Nhấn F10
Chạy vào hàm	Nhấn F11
Xem giá trị biến	Đưa chuột vào biến muốn xem giá trị

1.3 NỘI DUNG THỰC HÀNH:

1.3.1 Câu 1:

Cho đoạn mã sau, yêu cầu sinh viết và chạy chương trình.

- File **`thuvien_lab1.h`** như sau:

Dòng	Mã lệnh
1	`#include<iostream>`
2	`using namespace std;`
3	`const int MAX = 50;`
4	`//Cấu trúc dãy số`
5	`struct DaySo{`
6	`int ds[MAX];`
7	`int n;  //Số phần tử trong dãy số`
8	`};`
9	`void khoiTao(DaySo &day);`
10	`int kiemTraRong(DaySo day);`
11	`int kiemTraDay(DaySo day);`
12	`int laySoPhanTu(DaySo day);`
13	`void docDanhSach(DaySo &day);`
14	`void xuatDanhSach(DaySo day);`

- File **caidat_lab1.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab1.h"`
2	`//Hàm khoi tao`
3	`void khoiTao(DaySo &day){`
4	` day.n = 0;`
5	`}`
6	`//Ham kiem tra rong`
7	`bool kiemTraRong(DaySo day){`
8	` return (day.n == 0 ? 1 : 0);`
9	`}`
10	`//Ham kiem tra day`
11	`bool kiemTraDay(DaySo day){`
12	` return (day.n == MAX ? 1 : 0);`
13	`}`
14	`//Ham tra ve so nut cua DS dac`
15	`int laySoPhanTu(DaySo day){`
16	` return day.n;`
17	`}`
18	`/********Ham nhap day so vao ds DAC***********/`
19	`void docDanhSach(DaySo &day){`
20	` cout << "\n Nhap vao so phan tu cua DS:";`
21	` cin >> day.n;`
22	` for (int i = 0; i<day.n; i++){`
23	`  cout << "\n Nhap vao phan tu thu " << i + 1 << ":";`
24	`  cin >> day.ds[i];`
25	` }`
26	`}`
27	`/********Ham xuat ra DS DAC***************/`
28	`void xuatDanhSach(DaySo day){`
29	` cout << "\n Phan tu trong DS:";`
30	` for (int i = 0; i<day.n; i++)`
31	`  cout << day.ds[i] << " ";`
32	`}`

Yêu cầu:

a/. Viết vào file **chuongtrinh_lab1.cpp** để gọi và chạy các hàm đã viết.

b/. Cho biết ý nghĩa của đoạn chương trình trên.

1.3.2 Câu 2:

Sau đây là đoạn mã thêm một phần tử vào dãy số ở vị trí bất kỳ, lời gọi hàm trong thân hàm, sinh viên hãy soạn thảo lại mã nguồn và chạy thử chương trình.

- File **caidat_lab1.cpp** sinh viên soạn đoãn mã sau vào:

Dòng	Mã lệnh
1	void chenViTri(DaySo &day, int viTri, int a){
2	if (viTri <0 || viTri > laySoPhanTu(day))
3	cout << "\n Vi tri " << viTri << " khong hop le";
4	else
5	if (kiemTraRong(day) == true){
6	if (viTri == 0){
7	day.ds[0] = a;
8	day.n++;
9	}
10	else
11	cout << "\n Vi tri khong hop le";
12	}
13	else if (kiemTraDay(day) == true)
14	cout << "\n Danh sach day";
15	else{
16	for (int i = laySoPhanTu(day) - 1; i >= viTri;
17	i--)
18	day.ds[i + 1] = day.ds[i];
19	day.ds[viTri] = a;
20	day.n++;
21	}
22	}

- ở file **chuongtrinh_lab1.cpp** như sau:

Dòng	Mã lệnh
1	#include"thuvien_lab1.h"
2	void main(){
3	DaySo day;
4	int viTri, a, k;
5	khoiTao(day);
6	docDanhSach(day);
7	xuatDanhSach(day);
8	cout << "\n Nhap phan tu can chen:";
9	cin >> k;
10	cout << "\n Nhap vi tri can chen:";
11	cin >> viTri;
12	chenViTri(day, viTri, k);
13	xuatDanhSach(day);
14	}

1.3.3 Câu 3:

Cho đoạn mã xóa phần tử ở vị trí, yêu cầu sinh viên viết lại và gọi hàm trong void main ().

Dòng	Mã lệnh

1
2
3
4
5
6
7
8
9
10
11
12
13
	void xoaViTri(DaySo &day, int viTri){
	int i;
	if (viTri < 0 || viTri > laySoPhanTu(day) - 1)
	cout << "Vi tri" << viTri << "khong hop le !";
	else
	if (kiemTraRong(day)) cout << "DS bi rong";
	else{
	for (i = viTri + 1; i <= laySoPhanTu(day);
	i++)
	day.ds[i - 1] = day.ds[i];
	day.n--;
	}
	}

1.3.4 Câu 4:

Sau đây là hàm xóa một phần tử có giá trị bằng k, có trong dãy số. Nhưng hàm **đang có lỗi sai**, yêu cầu sinh viên sửa lại hàm cho đúng. Viết void main () gọi hàm trên.

Dòng	Mã lệnh

1
2
3
4
5
6
7
8
9
	void xoaGiaTriK(DaySo &day, int k){
	for (int i = 0; i<day.n; i++){
	if (day.ds[i] == k){
	for (int j = i; j< day.n - 1; j++)
	day.ds[j] = day.ds[j - 1];
	day.n--;
	}
	}
	}

1.3.5 Câu 5:

Sinh viên tự viết chương trình để thực hiện các công việc sau:

a/. Nhập/xuất danh sách có n phần tử là số nguyên.

b/. Xuất số lượng phần tử có giá trị là x trong dãy số *(x là số nhập vào khi chạy chương trình)*.

c/. Xuất lần lượt tất cả vị trí của phần tử có giá trị là x trong dãy số. *(x là phần tử được nhập vào)*

1.4 BÀI TẬP VỀ NHÀ

1.4.1 Câu 1:

Đếm số lần xuất hiện của giá trị X có trong dãy số.

1.4.2 Câu 2:

Tìm vị trí xuất hiện của phần tử X có trong dãy số A. Thay thế giá trị X đó thành giá trị Y.

1.4.3 Câu 3:

Viết hàm chèn phần tử X, vào vị trí k có trong dãy số.

1.4.4 Câu 4:

Sinh viên tự viết chương trình sau:

a/. Ghép hai dãy số lại với nhau thành 1 dãy số kết quả. Giá trị các phần tử trong dãy số kết quả có nội dung là giá trị các phần tử của dãy số 1 rồi đến dãy số 2.

b/. Tách dãy số ban đầu thành 2 dãy số: một dãy chứa phần tử chẵn, một dãy chứa phần tử lẻ.

2 GIẢI THUẬT TÌM KIẾM – TRÊN SỐ NGUYÊN & TRÊN ĐỐI TƯỢNG

2.1 MỤC TIÊU:

Trong bài thực hành này, sinh viên sẽ:

- Tìm kiếm tuyến tính
  - Trên số nguyên
  - Trên đối tượng (sinh viên, môn học,..)
- Tìm kiếm nhị phân
  - Trên số nguyên
  - Trên đối tượng (sinh viên, môn học,..)

2.2 TÓM TẮT LÝ THUYẾT:

- Khai báo cấu trúc	`struct <tên kiểu cấu trúc>{ <kiểu dữ liệu 1> tên thành phần 1;`
	`...`
	`<kiểu dữ liệu n> tên thành phần n; };`

- Một số lưu ý:	- Bắt buộc phải có **;** sau dấu **}** - Tên của kiểu struct **nên** bắt đầu bằng ký tự hoa. Các thành phần thuộc tính cùng kiểu dữ liệu nên cách nhau dấu , như các điểm toán, lý, hóa.
- Truy xuất biến kiểu cấu trúc	- **Đối với thành phần là 1 kiểu dữ liệu cơ bản:** <tên biến cấu trúc>.<tên thành phần> - **Đối với thành phần là 1 kiểu cấu trúc:** <tên biến cấu trúc 1>. <tên biến cấu trúc 2>.<tên thành phần>

- Ví dụ khai báo cấu trúc sinh viên gồm có: Mã sinh viên, họ tên, điểm toán, lý, hóa, trung bình.	`struct SinhVien{`
	`char maSinhVien[10];`
	`char hoTen[30];`
	`float toan,ly,hoa,dtb;`

	};

Tìm kiếm tuần tự	int timKiemTuanTu(DaySo x,int a) {
	int i=0;
	while(i<day.n && day.ds[i]!=a)
	i++;
	if (i==day.n)
	return -1 ;//Khong tim thay return i;
	}

Tìm kiếm nhị phân:	//Tim kiem nhi phan
	Int timKiemNhiPhan(DaySo day, int a)
	{
	int bottom=0, top=day.n-1; int mid;
	
	do{
	mid=(bottom+top)/2;
	if(a == day.ds[mid]) return mid;
	else
	if (a<day.ds[mid]) top=mid-1;
	else bottom= mid+1;
	} while (bottom<=top);
	return -1;
	}

2.3 NỘI DUNG THỰC HÀNH:

2.3.1 Tìm kiếm tuần tự trên dãy số

2.3.1.1 Cho đoạn mã sau, yêu cầu sinh viên viết và chạy chương trình.

- File **thuvien_lab2_1.h** như sau:

Dòng	Mã lệnh
1	#include<iostream>
2	using namespace std;
3	const int MAX = 50;
4	//Cấu trúc Dãy số
5	struct DaySo{
6	int ds[MAX];
7	int n; //số phần tử trong dãy số
8	};

```
9   void khoiTao(DaySo &day);
10  int kiemTraRong(DaySo day);
11  int kiemTraDay(DaySo day);
12  int laySoPhanTu(DaySo day);
13  void docDanhSach(DaySo &day);
14  void xuatDanhSach(DaySo day);
```

- File **caidat_lab2_1.cpp** như sau:

Dòng	Mã lệnh

```
1   #include"thuvien_lab2_1.h"
2   //Hàm khoi tao
3   void khoiTao(DaySo &day){
4         day.n = 0;
5   }
6   //Ham kiem tra rong
7   int kiemTraRong(DaySo day){
8         return (day.n == 0 ? 1 : 0);
9   }
10  //Ham kiem tra day
11  int kiemTraDay(DaySo day){
12        return (day.n == MAX ? 1 : 0);
13  }
14  //Ham tra ve so nut cua DS dac
15  int laySoPhanTu(DaySo day){
16        return day.n;
17  }
18  /********Ham nhap day so vao ds DAC***********/
19  void docDanhSach(DaySo &day){
20        cout << "\n Nhap vao so phan tu cua DS:";
21        cin >> day.n;
22        for (int i = 0; i<day.n; i++){
23              cout << "\n Nhap vao phan tu thu " << i + 1 << ":";
24              cin >> day.ds[i];
25        }
26  }
27  /********Ham xuat ra DS DAC***************/
28  void xuatDanhSach(DaySo day){
29        cout << "\n Phan tu trong DS:";
30        for (int i = 0; i<day.n; i++)
31              cout << day.ds[i] << " ";
32  }
```

Yêu cầu:

c/. Viết vào file **chuongtrinh_lab2_1.cpp** để gọi và chạy các hàm đã viết.

d/. Cho biết ý nghĩa của đoạn chương trình trên.

2.3.2 Gọi hàm

Đoạn mã dưới đây là hàm tìm kiếm tuần tự, cách gọi hàm tìm kiếm tuần tự trong **void main.**

Yêu cầu:

- Sinh viên soạn thảo và chạy chương trình.
- Sửa lại hàm main, để xuất ra vị trí tìm thấy phần tử a cần tìm ở vị trí thứ mấy có trong DS. Ngược lại thì xuất ra không tìm thấy.

Dòng	Mã lệnh
1	`int timKiemTuanTu(DaySo day, int x){`
2	`  int i = 0;`
3	`  while (i<day.n && day.ds[i] != x)`
4	`  i++;`
5	`  if (i == day.n)`
6	`  return -1;//Khong tim thay`
7	`  return i;`
8	`}`

- File **chuongtrinh_lab2_1.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab2_1.h"`
2	`void main(){`
3	`  DaySo day;`
4	`  int k;`
5	`  khoiTao(day);`
6	`  docDanhSach(day);`
7	`  xuatDanhSach(day);`
8	`  cout << "\n Nhap phan tu k can tim:";`
9	`  cin >> k;`
10	`  if (timKiemTuanTu(day, k) == -1)`
11	`  cout << "\n Khong tim thay";`
12	`  else`
13	`  cout << "\n Tim thay";`
14	`}`

2.3.3 Câu 2: Tìm kiếm nhị phân trên dãy số

Đoạn mã dưới đây là hàm tìm kiếm nhị phân, cách gọi hàm tìm kiếm nhị phân trong **`void main.`**

Yêu cầu:

- Sinh viên soạn thảo và chạy chương trình.
- Sửa lại hàm main, để xuất ra vị trí tìm thấy phần tử a cần tìm ở vị trí thứ mấy có trong DS. Ngược lại thì xuất ra không tìm thấy.

Dòng	Mã lệnh
1	`int timKiemNhiPhan(DaySo day, int a){`
2	`    int bottom=0, top=day.n-1; int mid;`
3	
4	`    do{`
5	`        mid=(bottom+top)/2;`
6	`        if(a == day.ds[mid]) return mid;`
7	`        else`
8	`            if (a<day.ds[mid]) top=mid-1;`
9	`            else bottom= mid+1;`
10	`    } while (bottom<=top);`
11	`    return -1;`
12	
13	`}`

- File **`chuongtrinh_lab2_1.cpp`** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab2_1.h"`
2	`void main(){`
3	`    DaySo day;`
4	`    int x;`
5	`    khoiTao(day);`
6	`    docDanhSach(day);`
7	`    xuatDanhSach(day);`
8	`    cout << "\n Nhap phan tu x can tim:";`
9	`    cin >> x;`
10	`    if (timKiemNhiPhan(day, x) == false)`
11	`        cout << "\n Khong tim thay";`
12	`    else`
13	`        cout << "\n Tim thay";`
14	

	}

2.3.4 Câu 3: Tìm tuần tự trên đối tượng

2.3.4.1 *Câu 3.1:*

Đoạn mã sau dùng để khai báo cấu trúc một môn học, có hàm nhập, xuất ra một môn học:

- Trong file **`thuvien_lab2_2.h`** có đoạn mã như sau:

Dòng	Mã lệnh

1
2
3
4
5
6
7
8
9
10
11
12
	`#include<iostream>`
	`#include<string.h>`
	`using namespace std;`
	`const int MAX = 50;`
	`struct MonHoc{`
	`  char maMon[10];`
	`  char tenMon[30];`
	`  int soTinChi;`
	`};`
	`void nhapMonHoc(MonHoc &mh);`
	`void xuatMonHoc(MonHoc mh);`

- Trong file **`caidat_lab2_2.cpp`**, đoạn mã như sau:

Dòng	Mã lệnh

1
2
3
4
5
6
7
8
9
10
11
12
13
14
	`#include"thuvien_lab2_2.h"`
	`void nhapMonHoc(MonHoc &mh){  cout << "\n NHap ma mh:";`
	`  cin >> mh.maMon;`
	`  cout << "\n NHap Ten MH:";`
	`  cin.ignore();`
	`  cin.getline(mh.tenMon, 30);`
	`  cout << "\n Nhap so tin chi:";`
	`  cin >> mh.sotc;`
	`}`
	`void xuatMonHoc(MonHoc mh){`
	`  cout << "Ma MH" << "\t" << "TenMH" << "\t\t" << "So TC" << endl;`
	`  cout  << mh.maMon << "\t" << mh.tenMon << "\t\t" << mh.soTinChi<<endl;`

```
15  }
16
```

Trong file **chuongtrinh_lab2_2.cpp**, đoạn mã như sau:

Dòng	Mã lệnh

```
1
2   #include"thuvien_lab2_2.h"
3   void main(){
4         MonHoc mh;
5         nhapMonHoc(mh);
6         cout << "\n Mon hoc ban vua nhap:" << endl;
7         xuatMonHoc(mh);
8   }
```

Yêu cầu:

- Sinh viên nhập đoạn mã trên và chạy chương trình xuất ra kết quả như sau:

```
C:\WINDOWS\system32\cmd.exe

NHap ma mh:mh01

NHap Ten MH:cautrucdl

Nhap so tin chi:3

Mon hoc ban vua nhap:mh01       cautrucdl       3
```

2.3.5 Câu 3.2:

Sau đây là cấu trúc của một danh sách môn học.

- Trong file **thuvien_lab2_2.h** có đoạn mã như sau:

Dòng	Mã lệnh

```
1
2   #include<iostream>
3   #include<string.h>
4   using namespace std; const
5   int MAX = 50; struct MonHoc{
6         char maMon[10];
7         char tenMon[30];
8       int soTinChi;
9   };
10  struct DanhSachMonHoc{
11      int soMon;
12     MonHoc danhSach[MAX];
13  };
14  void nhapMonHoc(MonHoc &mh);
15  void xuatMonHoc(MonHoc mh);
16  void nhapDanhSachMonHoc(DanhSachMonHoc &ds);
17  void xuatDanhSachMonHoc(DanhSachMonHoc ds);
18
```

- Trong file **caidat_lab2_2.cpp**, đoạn mã như sau:

Dòng	Mã lệnh

```
1
2   void nhapDanhSachMonHoc(DanhSachMonHoc &ds){
3         cout << "\n Nhap so luong mon hoc:";
4         cin >> ds.soMon;
5         cout << "\n Nhap thong tin tung mon hoc:";
6         for (int i = 0; i<ds.soMon; i++){
7             cout << "\n Thong tin Mon Hoc Thu:" << i + 1 << endl;
8             nhapMonHoc(ds.danhSach[i]);
9         }
10  }
11  void xuatDanhSachMonHoc(DanhSachMonHoc ds){
12        cout << "\n Danh sach cac mon hoc:" << endl;
13        cout << "\n MaMH \t TenMH \t so tc"<<endl;
14        for (int i = 0; i<ds.soMon; i++)
15            xuatMonHoc(ds.danhSach[i]);
16  }
17
```

- Trong file **chuongtrinh_lab2_2.cpp**, đoạn mã như sau:

Dòng	Mã lệnh

1
2
3
4
5
6
7
	`#include"thuvien_lab2_2.h"`
	`void main(){`
	`  DanhSachMonHoc ds;`
	`  nhapDanhSachMonHoc(ds);`
	`  xuatDanhSachMonHoc(ds);`
	`}`

Yêu cầu:

- Sinh viên bổ sung vào các hàm **`nhapMonHoc`** để nhập một môn học, **`xuatMonHoc`** để xuất một môn học cho chương trình trên có thể chạy được.

Các bạn hãy nhập vào danh sách 2 môn học để chạy thử.

2.3.6 Câu 3.3:

Dưới đây là đoạn mã của hàm tìm kiếm, nhập vào mã môn học, sử dụng cấu trúc tìm kiếm tuần tự áp dụng trên cấu trúc danh sách môn học để tìm ra môn học thỏa mãn điều kiện nhập vào.

- Trong file **`caidat_lab2_2.cpp`**, đoạn mã như sau:

Dòng	Mã lệnh

1
2
3
4
5
6
7
8
9
	`int timKiemTuanTu(DanhSachMonHoc ds, char* maCanTim){`
	`  int i = 0;`
	`  while (i<ds.so && strcmp(ds.data[i].maMon, maCanTim) != 0)`
	`  i++;`
	`  if (i == ds.so)`
	`  return -1;//Khong tim thay`
	`  return i;`
	`}`

- Trong file **`chuongtrinh_lab2_2.cpp`**, đoạn mã như sau:

Dòng	Mã lệnh

```
1   #include"thuvien_lab2_2.h"
2   void main(){
3         DanhSachMonHoc ds;
4         nhapDanhSachMonHoc(ds);
5         xuatDanhSachMonHoc(ds);
6         char maCanTim[10];
7         cout << "\n Nhap ma MH muon tim:";
8         cin >> maCanTim;
9         int k = timKiemTuanTu(ds, maCanTim);
10        if (k == -1)
11              cout << "\n Khong co ma mh trong DS";
12        else{
13              cout << "\n Co mon hoc ban can tim:";
14              xuatMonHoc(ds.data[k]);
15        }
16  }
17
```

Yêu cầu:

a/. Nhập đoạn mã trên và chạy chương trình.

b/. Giải thích lệnh **`strcmp(ds.data[i].maMonHoc, ma)`** ở đoạn mã trên

2.3.7 Câu 4: Tìm nhị phân trên đối tượng

Sinh viên hãy viết hàm tìm môn học bằng thuật toán tìm nhị phân và vào hàm main gọi hàm tìm nhị phân đó.

2.4 BÀI TẬP VỀ NHÀ:

2.4.1 Câu 1:

Đếm số lần xuất hiện của môn học X có trong danh sách môn học.

2.4.2 Câu 2:

Tìm vị trí xuất hiện của môn học có mã là X có trong danh sách môn học. Thay thế môn học có mã X đó thành môn học có mã Y.

2.4.3 Câu 3:

Cho người dùng nhập số tín chỉ. Viết hàm tìm tất cả môn học có số tín chỉ bằng số tín chỉ nhập vào. (tìm tuyến tính)

3 DANH SÁCH LIÊN KẾT ĐƠN

3.1 MỤC TIÊU

- Khai báo danh sách liên kết đơn
- Viết chương trình thể hiện các tác vụ lên danh sách: cài đặt, khởi tạo, duyệt, thêm phần tử, loại bỏ phần tử, thay thế giá trị, ghép, trộn danh sách.
  - ➢ Trên số nguyên
  - ➢ Trên đối tượng

3.2 TÓM TẮT LÝ THUYẾT

<table>
<tr><th>Nội dung</th><th>Ví dụ</th></tr>
<tr><td colspan="2">1. Danh sách liên kết đơn:
• Định nghĩa : DSLK là danh sách mà các phần tử được nối kết với nhau nhờ vào vùng liên kết của chúng.
• Danh sách liên kết đơn là danh sách liên kết trong đó mỗi phần tử chỉ có một liên kết với phần tử tiếp theo.
• Cách biểu diễn: Mỗi phần tử của danh sách liên kết đơn gồm hai vùng:
- Vùng chứa thông tin <u>data</u>
- Vùng chứa liên kết đến phần tử kế <u>link</u>
• Hình ảnh minh họa:</td></tr>
<tr><td>- Khai báo kiểu dữ liệu cho danh sách số nguyên:</td><td><pre>typedef int TYPEINFO;
struct Node{
    TYPEINFO data;
    Node* link;
};
typedef Node* NodePtr;
Typedef NodePtr list;</pre></td></tr>
<tr><td>- Khởi tạo danh sách liên kết</td><td><pre>void init(NodePtr &list){
    list =NULL;

}</pre></td></tr>
<tr><td>- Hàm tạo một node:</td><td><pre>NodePtr createNode(TYPEINFO x){
    NodePtr p;
    p=new Node;
    p->data =x;
    p->link=NULL;
    return p;</pre></td></tr>
</table>

	}

- Hàm giải phóng bộ nhớ	void freeList(NodePtr &list){
	NodePtr p=list;
	while(p!=NULL){
	list=list->link;
	delete p;
	p=list;
	}
	}

- Kiểm tra danh sách rỗng hay không	int isEmpty(NodePtr list){
	return(list==NULL ? 1:0);
	}

- Thêm 1 phần tử vào đầu DSLK đơn:	NodePtr addFirst(NodePtr &list,TYPEINFO x){
	NodePtr p;
	p=createNode(x);
	p->link=list;
	list=p;
	return p;
	}

- Thêm 1 phần tử vào giữa DSLK đơn (Thêm sau phần tử q):	NodePtr addAfterQ(NodePtr q, TYPEINFO x){
	NodePtr p=createNode(x);
	p->link = q->link;
	q->link=p;
	return q;
	}

- Thêm 1 phần tử vào cuối DSLK đơn:	//Ham Them CUOI
	NodePtr addLast(NodePtr &list, TYPEINFO x){
	NodePtr p;
	p=createNode(x);
	if(list==NULL)
	list=p;
	else{
	NodePtr q=list;
	while(q->link!=NULL)
	q = q->link;
	q->link=p;
	}
	return p;

	}

- Hàm xóa 1 phần tử ở đầu DSLK đơn	void deleteFirst(NodePtr &list){
	NodePtr p;
	if(list!=NULL){
	p=list;
	list=p->link;
	delete p;
	}
	}

- Hàm xóa 1 phần tử có giá trị là x chứa trong DSLK đơn	void deleteX(NodePtr &list, int x){
	NodePtr p, before;
	if(list->data ==x)
	deleteFirst(list);
	else{
	p=list;
	before =list;
	while((p!=NULL)&&(p->data!=x)){
	before =p;
	p=p->link;
	}
	if(p!=NULL){
	before->link=p->link;
	delete p;
	}
	}
	}

- Hàm xóa 1 phần tử ở cuối trong DSLK đơn	void deleteLast(NodePtr &list)
	{
	NodePtr p, before;
	p=list;
	before=list;
	if(p->link==NULL){
	delete p;
	list=NULL;
	}
	else{
	while(p->link!=NULL){
	before=p;
	p=p->link;
	}
	if(p->link==NULL){

```
                    delete p;
                    before->link=NULL;
                }
            }
        }
```

3.3 NỘI DUNG THỰC HÀNH

3.3.1 Câu 1:

Dưới đây là đoạn mã nguồn dùng để nhập dữ liệu cho một danh sách liên kết.

- Trong file **thuvien_lab3.h**, đoạn mã như sau:

Dòng	Mã lệnh
1	`#include<iostream>`
2	`using namespace std;`
3	`typedef int TYPEINFO;`
4	
5	`struct Node{`
6	`    TYPEINFO data;`
7	`    Node *link;`
8	`};`
9	`typedef Node *NodePtr;`
10	`void init(NodePtr &list);`
11	`int isEmpty(NodePtr list);`
12	`NodePtr createNode(TYPEINFO x);`
13	`NodePtr addFirst(NodePtr &list, TYPEINFO x);`
14	`void createList(NodePtr &list);`
15	`void printList(NodePtr list);`
16	`void freeList(NodePtr &list);`

- Trong file **caidat_lab3.cpp**, đoạn mã như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab3.h"`
2	`void init(NodePtr &list){`
3	`    list = NULL;`
4	`}`
5	`int isEmpty(NodePtr list){`
6	`    return (list == NULL ? 1 : 0);`
7	`}`
8	`NodePtr createNode(TYPEINFO x){`
9	`    NodePtr p;`
10	`    p = new Node;`
11	`    p->data = x;`

```
12      p->link = NULL;
13      return p;
14  }
15  NodePtr addFirst(NodePtr &list, TYPEINFO x){
16      NodePtr p;
17      p = createNode(x);
18      p->link = list;
19      list = p;
20      return p;
21  }
22  void createList(NodePtr &list)
23  {
24      init(list);
25      int x;
26      do{
27          cout << "\n Enter x = (exit -99)";
28          cin >> x;
29          if (x == -99)
30              break;
31          addFirst(list, x);//Gọi hàm thêm đầu
32      } while (1);
33  }
34  void printList(NodePtr list){
35      NodePtr p = list;
36      while (p != NULL){
37          cout << p->data << " ";
38          p = p->link;
39      }
40  }
41  void freeList(NodePtr &list){
42      NodePtr p = list;
43      while (p != NULL){
44          list = list->link;
45          delete p;
46          p = list;
47      }
48  }
```

- Trong file **chuongtrinh_lab3.cpp**, đoạn mã như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab3.h"`
2	`void main(){`
3	`    NodePtr list;`
4	`    init(list);`

```
5       createList(list);
6       cout << "\n List:";
7       printList(list);
8       freeList(list);
9       system("pause");
10  }
```

Yêu cầu:

a. Sinh viên biên dịch và cho biết danh sách được tạo sử dụng kỹ thuật chèn đầu hay chèn cuối.

b. Cho biết ý nghĩa hàm **`void freeList(NodePtr &list).`**

c. Ghi kết quả xuất ra khi nhập vào dãy số sau**: 10, 8, 7, 9, 20.**

3.3.2 Câu 2:

Sinh viên viết lại hàm nhập dữ liệu cho danh sách liên kết sử dụng kỹ thuật chèn cuối.

3.3.3 Câu 3:

Sau đây là đoạn mã tìm kiếm phần tử có giá trị x có trong danh sách liên kết. Nếu tìm thấy trả về node đấy, nếu không trả về NULL.

- Trong file **`caidat_lab3.cpp`** như sau:

Dòng	Mã lệnh
1	`NodePtr findX(NodePtr list, TYPEINFO x){`
2	`  NodePtr p = list;`
3	`  while (p != NULL){`
4	`  if (p->data == x)`
5	`  return p;`
6	`  p = p->link;`
7	`  }`
8	`  return p;`
9	`}`

Yêu cầu:

- Sinh viên hãy gọi hàm vừa viết ở trên trong **`void main.`**

3.3.4 Câu 4:

Đoạn mã sau là chèn node có dữ liệu là x vào sau node do con trỏ q quản lý, đoạn mã như sau:

Dòng	Mã lệnh

```
1  NodePtr addAfterQ(NodePtr q, TYPEINFO x)
2  {
3        NodePtr p =createNode(x);
4        p->link = q->link;
5        q->link = p;
6        return q;
7  }
```

Yêu cầu:

- Viết các hàm để hoàn thiện chèn phần tử có nội dung là x được thêm vào sau phần tử do con trỏ q quản lý có trong danh sách liên kết.

3.3.5 Câu 5:

Sau đây là đoạn mã xóa đầu, sinh viên nhập và chạy chương trình.

- Trong file **caidat_lab3.cpp** như sau:

Dòng	Mã lệnh
1	`void deleteFirst(NodePtr &list)`
2	`{`
3	`  NodePtr p;`
4	`  if (list != NULL){`
5	`  p = list;`
6	`  list = p->link;`
7	`  delete p;`
8	`  }`
9	`}`

- Trong file **chuongtrinh_lab3.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab3.h"`
2	`void main(){`
3	`  NodePtr list;`
4	`  init(list);`
5	`  createList(list);`
6	`  cout << "\n List:";`
7	`  printList(list);`
8	`  deleteFirst(list);`
9	`  cout << "\n List after deleting first item:";`
10	`  printList(list);`
11	`  freeList(list);`
12	`  system("pause");`
13	

	}

Yêu cầu:

- Viết hàm xóa node cuối của danh sách liên kết và chạy chương trình.

3.3.6 Câu 6:

Viết hàm xóa node có nội dung là x có trong DSLK, biết nguyên mẫu hàm như sau:

```
void deleteX(NodePtr &list, int x)
```

Ví dụ: có danh sách liên kết gồm các node sau: 5, 8, -12, 9, 18

Phần tử x cần xóa là: -12.

Xuất ra danh sách liên kết sau khi xóa node x là: 5, 8, 9, 18

3.3.7 Câu 7:

Viết hàm đảo ngược một DSLk.

Ví dụ: danh sách liên kết ban đầu: **9 -10 25 11 -9**

- Sau khi đảo ngược ta được danh sách: **-9, 11, 25, -10, 9**

3.4 BÀI TẬP VỀ NHÀ

3.4.1 Câu 1:

Viết hàm tách một DSLK ra thành 2 DS, một DSLK chứa phần tử chẵn, một DSLK chứa phần tử lẻ.

Ví dụ: Danh sách ban đầu: 9, -8, 7, 11, 20, -2, 15

Xây dựng hàm tách danh sách liên kết ta được:

- **Danh sách 1 chứa phần tử là các số chẵn**: -8 20 -2
- **Danh sách 2 chứa phần tử là các số lẻ**: 9 7 11 15

3.4.2 Câu 2:

Viết hàm xóa tất cả các phần tử có nội dung là x có trong DSLK, biết nguyên mẫu hàm:

```
void deleteAllX(NodePtr &list,int x)
```

Ví dụ: danh sách ban đầu: **9 11 5 9 2 9 -17**

- Phần tử cần xóa **x = 9**
- Danh sách còn lại sau khi xóa tất cả các phần tử 9:

11 5 2 -17

4 DANH SÁCH LIÊN KẾT ĐƠN – TRÊN ĐỐI TƯỢNG

4.1 MỤC TIÊU:

Khai báo danh sách liên kết đơn cho đối tượng

Viết chương trình thể hiện các tác vụ lên danh sách: cài đặt, khởi tạo, duyệt, thêm phần tử, loại bỏ phần tử, thay thế giá trị, ghép, trộn danh sách trên đối tượng.

4.2 TÓM TẮT LÝ THUYẾT

Nội dung	Ví dụ
1. Cấu trúc	

- Khai báo cấu trúc	`struct <tên kiểu cấu trúc>{`
	`<kiểu dữ liệu 1> tên thành phần 1;`
	`...`
	`<kiểu dữ liệu n> tên thành phần n;`
	`};`

- Một số lưu ý:	- Bắt buộc phải có ; sau dấu } - Tên của kiểu struct nên bắt đầu bằng ký tự hoa. - Các thành phần thuộc tính cùng kiểu dữ liệu nên cách nhau dấu , như các điểm toán, lý, hóa.
- Truy xuất biến kiểu cấu trúc	**- Đối với thành phần là 1 kiểu dữ liệu cơ bản:** <tên biến cấu trúc>.<tên thành phần> **- Đối với thành phần là 1 kiểu cấu trúc:** <tên biến cấu trúc 1>. <tên biến cấu trúc 2>.<tên thành phần>

- Ví dụ khai báo cấu trúc sinh viên gồm có các thành phần sau: masv, hotensv, namsinh, toan, ly, hoa, dtb.	`struct Student{`
	`  char id[10];`
	`  char fullName[30];`
	`  float score;`
	`};`

- **Khai báo cấu trúc danh sách sinh viên**	`typedef struct Node* NodePtr;`
	`struct Node{`
	`  Student data;`
	`  NodePtr link;`
	`};`

4.3 NỘI DUNG THỰC HÀNH

4.3.1 Câu 1:

Đoạn mã sau dùng cấu trúc SinhVien gồm có **`id, fullName, score`** để nhập vào một danh sách sinh viên, và xuất danh sách sinh viên vừa nhập:

- Trong file **`thuvien_lab4.h`** như sau:

Dòng	Mã lệnh
1	`#define _CRT_SECURE_NO_WARNINGS`
2	`#include<iostream>`
3	`#include<iomanip>`
4	`#include"string.h"`
5	`using namespace std;`
6	`typedef struct Node* NodePtr;`
7	`struct Student{`
8	`  char id[10];`
9	`  char fullName[30];`
10	`  float score;`
11	`};`
12	`struct Node{`
13	`  Student data;`
14	`  NodePtr link;`
15	`};`
16	`void init(NodePtr& list);`
17	`int isEmpty(NodePtr list);`
18	`void freeList(NodePtr& list);`
19	`NodePtr createStudent(Student x);`
20	`NodePtr addFirst(NodePtr& list, Student x);`
21	`bool isDuplicate(NodePtr& list, char* id);`
22	`void createList(NodePtr &list);`
23	`void printList(NodePtr list);`

- Trong file **`caidat_lab4.cpp`** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab4.h"`
2	`void init(NodePtr& list){`
3	`  list = NULL;`

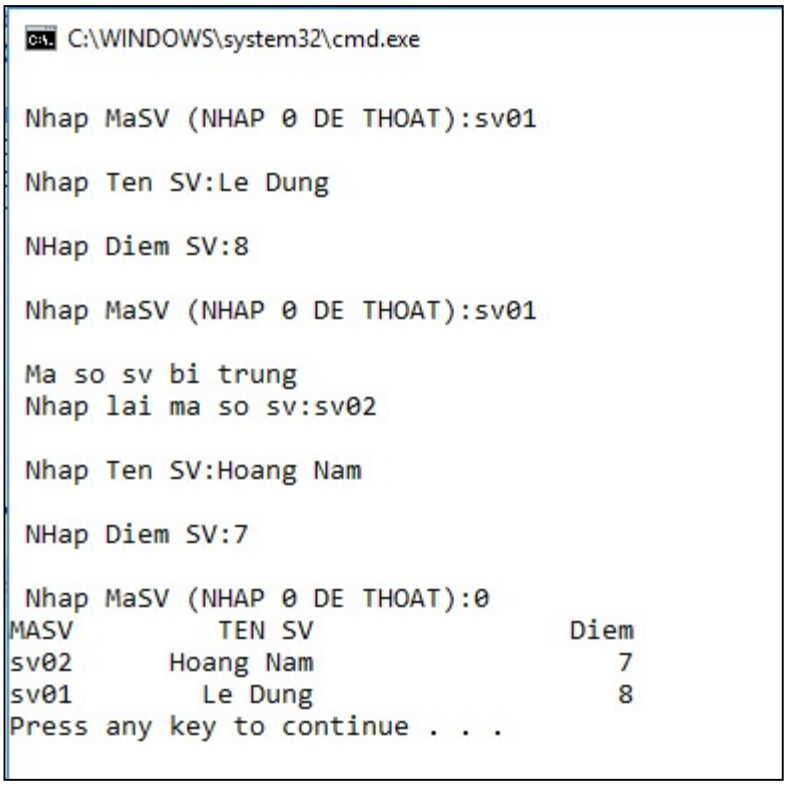
```
4   }
5   int isEmpty(NodePtr list){
6         return list == NULL ? 1 : 0;
7   }
8   void freeList(NodePtr& list){
9         NodePtr p = list;
10        while (p != NULL){
11              list = list->link;
12              delete p;
13              p = list;
14        }
15  }
16  NodePtr createStudent(Student x){
17        NodePtr p;
18        p = new Node;
19        p->data = x;
20        p->link = NULL;
21        return p;
22  }
23  //Ham Them 1 sinh vien
24  NodePtr addFirst(NodePtr &list, Student x)
25  {
26        NodePtr p = createStudent(x);
27        p->link = list;
28        list = p;
29        return p;
30  }
31  bool isDuplicate(NodePtr& list, char* id){
32        NodePtr p = list;
33        while (p != NULL){
34              if (strcmp(p->data.id, id) == 0)
35                    return true;
36              p = p->link;
37        }
38        return false;
39  }
40  //Nhap DSSV
41  void createList(NodePtr &list){
42        //init(list);
43        ////char  x[10];
44        Student student;
45        init(list);
46        do{
47              cout << "\n Enter id (enter 0 to exit):";
48              cin.getline(student.id, 10);
```

```
49              while (isDuplicate(list, student.id)){
50                    cout << "\n Id is duplicate";
51                    cout << "\n Enter id again,please:";
52                    cin.getline(student.id, 10);
53              }
54              if (strcmp(student.id, "0") == 0)  //NHAP MA SO = 0
55    DE THOAT
56                    break;
57              cout << "\n Enter fullname:";
58              cin.getline(student.fullName, 30);
59              cout << "\n Enter score:";
60              cin >> student.score;
61              cin.ignore();
62              addFirst(list, student);
63        } while (1);
64    }
65    void printList(NodePtr list){
66        cout << setw(0) << "Id" << setw(15) << "Fullname" <<
67    setw(20) << "Score" << endl;
68        while (list != NULL){
69              cout << setw(0) << list->data.id << setw(15) <<
70    list->data.fullName << setw(20) << list->data.score << endl;
71              list = list->link;
72        }
73    }
```

- Trong file **chuongtrinh_lab4.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab4.h"`
2	`void main(){`
3	`    NodePtr list;`
4	`    createList(list);`
5	`    printList(list);`
6	`    freeList(list);`
7	`    system("pause");`
8	`}`

Kết quả xuất ra ở đoạn mã trên như sau: (Hình ảnh chỉ mang tính chất minh họa)



4.3.2 Câu 2:

Yêu cầu, sinh viên sử dụng đoạn mã trên để viết hàm thêm cuối sinh viên, gọi và chạy lại chương trình.

4.3.3 Câu 3:

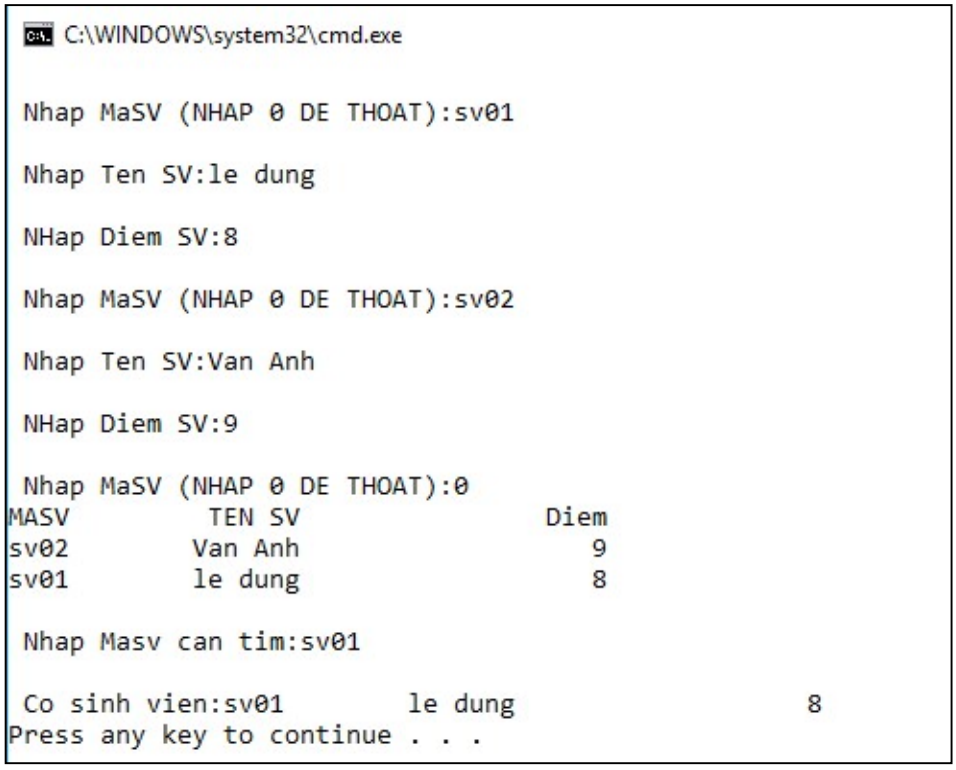
Cho hàm tìm kiếm một node trong DSLK:

Dòng	Mã lệnh
1	`NodePtr findXNode(NodePtr list, int x){`
2	`  NodePtr p = list;`
3	`  while (p != NULL){`
4	`  if (p->data == x)`
5	`  return p;`
6	`  p = p->link;`
7	`  }`
8	`  return p;`
9	`}`

Yêu cầu:

- Chỉnh sửa hàm **`findXNode`** ở trên, để xuất ra thông tin của sinh viên vừa tìm được dựa vào mã sinh viên nhập vào, có nguyên mẫu hàm như sau: **`NodePtr findStudentById(NodePtr list, Student x)`**

- **Kết quả xuất ra như sau (Hình ảnh chỉ mang tính chất minh họa):**



4.3.4 Câu 4:

Đoạn mã sau đây là hàm xóa đầu (**`deleteFirst`**) của một danh sách.

Dòng	Mã lệnh
1	`void deleteFirst(NodePtr& list){`
2	`  NodePtr p;`
3	`  if (list != NULL){`
4	`  p = list;`
5	`  list = p->link;`
6	`  delete p;`
7	`  }`
8	`}`

Yêu cầu:

a/. Sinh viên hãy gọi hàm xóa đầu (**`deleteFirst(NodePtr& list))`** vừa viết ở file cài đặt ở trên vào trong hàm **`void main.`** Xuất ra danh sách sinh viên còn lại sau khi gọi hàm xóa đầu.

b/. Sinh viên hãy viết thêm hàm xóa cuối (**`deleteLast(NodePtr& list))`**, sau đó gọi hàm xóa cuối vừa viết vào trong **`void main`** để thực thi chương trình.

4.3.5 Câu 5:

Đoạn mã sau đây là hàm xóa một sinh viên khi biết mã sinh viên đó.

Dòng	Mã lệnh

```
1   void deleteStudentById(NodePtr& list, Student student){
2         if (strcmp(list->data.id, student.id) == 0){
3               NodePtr p = list;
4               list = list->link;
5               delete p;
6         }
7         else{
8               NodePtr p, before;
9               p = list;
10              before = list;
11              while (p != NULL &&strcmp(p->data.id, sv.id) != 0){
12                    before = p;
13                    p = p->link;
14              }
15              if (p != NULL){
16                    before->link = p->link;
17                    delete p;
18              }
19        }
20  }
```

Yêu cầu:

a/. Viết lời gọi hàm **`deleteStudentById`** ở trên trong hàm main.

b/. Viết hàm xóa tất cả các sinh viên có điểm <5 (**Bài tập nâng cao – về nhà**)

Gợi ý:

- Sử dụng hàm tìm kiếm những sinh viên có điểm dưới
- Viết hàm tìm kiếm những sinh viên có điểm dưới 5
- Viết hàm xóa tất cả các sinh viên có điểm dưới 5

4.4 BÀI TẬP VỀ NHÀ

4.4.1 Câu 1:

Sinh viên hãy viết hàm đảo ngược một DSLK có nguyên mẫu hàm như sau:

`void reverseList(NotePtr &list).` viết thêm các lệnh cần thiết để chương trình có thể thực thi được.

Ví dụ : (Hình ảnh chỉ mang tính chất minh họa)

DS nhập vào là :

Mssv	Ten	Diem
A5130001	Nguyen Van Lai	8
A5130002	Tran Thi Mai	3
A5130003	Le Thi Dao	4

DS đảo ngược là :

Mssv	Ten	Diem
A5130003	Le Thi Dao	4
A5130002	Tran Thi Mai	3
A5130001	Nguyen Van Lai	8

4.4.2 Câu 2:

Sinh viên hãy viết hàm **splitList()** để tách một danh sách sinh viên thành hai danh sách sinh viên trong đó :

- Một danh sách sinh viên chứa các sinh viên có điểm nhỏ hơn 5
- Một danh sách sinh viên chứa các điểm lớn hơn hoặc bằng 5

Viết thêm các hàm cần thiết để minh họa hàm vừa viết thực thi được.

Ví dụ : **(Hình ảnh chỉ mang tính chất minh họa)**

DS nhập vào là :

Mssv	Tên	Điểm
A5130001	Nguyen Van Lai	8
A5130002	Tran Thi Mai	3

```
A5130003      Le Thi Dao        4
A5130004      Ta Thi Cuc        5
A5130005      Ly Thi Hue        8
A5130006      Huynh Kiem Lan    9
```

Tách thành 2 DS :

- ***DSSV có điểm số nhỏ hơn 5 :***

```
A5130002      Tran Thi Mai      3
A5130003      Le Thi Dao        4
```

- ***DSSV có điểm số lớn hơn hoặc bằng 5 :***

```
A5130001      Nguyen Van Lai    8
A5130004      Ta Thi Cuc        5
A5130005      Ly Thi Hue        8
A5130006      Huynh Kien Lan    8
```

4.4.3 Câu 3.

Nhập thêm vào cấu trúc sinh viên đã cho thông tin các điểm toán, lý, hóa. Yêu cầu tính điểm trung bình. Sắp xếp danh sách sinh viên theo chiều tăng dần của điểm trung bình.

5 DANH SÁCH LIÊN KẾT KÉP, LIÊN KẾT VÒNG

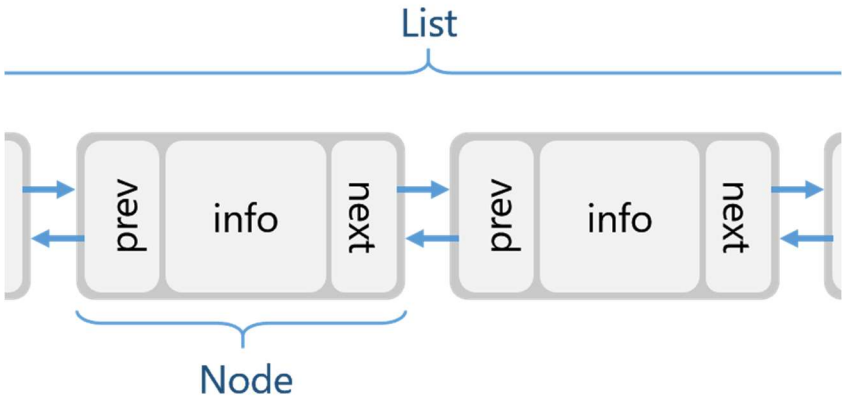
5.1 MỤC TIÊU

- Khai báo cấu trúc danh sách liên kết kép
- Viết chương trình thể hiện các tác vụ lên danh sách liên kết kép: cài đặt, khởi tạo, duyệt, thêm phần tử, loại bỏ phần tử, thay thế giá trị, ghép, trộn danh sách.
  - Trên số nguyên
  - Trên đối tượng
- Khai báo cấu trúc danh sách liên kết vòng
- Viết chương trình thể hiện các tác vụ lên danh sách liên kết vòng: cài đặt, khởi tạo, duyệt, thêm phần tử, loại bỏ phần tử, thay thế giá trị, ghép, trộn danh sách.
  - Trên số nguyên
  - Trên đối tượng

5.2 TÓM TẮT LÝ THUYẾT

1. Danh sách liên kết kép

Nội dung	Ví dụ
1. Danh sách liên kết KÉP (double linked list) • **Định nghĩa** : Danh sách liên kết kép cũng là một danh sách liên kết trong đó mỗi phần tử của nó liên kết với phần tử đứng trước và sau nó trong danh sách • **Cách biểu diễn**: Mỗi phần tử của danh sách liên kết đơn gồm 03 vùng: - Vùng chứa dữ liệu *(info)* - Vùng chứa liên kết đến phần tử kế *next* - Vùng chứa liên kết đến phần tử trước đó *pre* • **Hình ảnh minh họa:**	



- Khai báo kiểu dữ liệu cho danh sách số nguyên:

```
#include<iostream>
using namespace std;
typedef int TYPEINFO;
struct Node{
	TYPEINFO data;
	Node* next;
	Node* pre;
};
typedef Node* NodePtr;
struct DList{
	NodePtr first;
	NodePtr last;
};
```

- Khởi tạo danh sách liên kết

```
void init(DList& list){
	list.first = NULL;
	list.last = NULL;
}
```

- Hàm tạo một node:

```
NodePtr createNode(TYPEINFO x){
	NodePtr p;
	p = new Node;
	p->data = x;
	p->next = NULL;
	p->pre = NULL;
	return p;
}
```

- Kiểm tra danh sách rỗng hay không

```
int isEmpty(DList list){
	return (list.first == NULL ? 1 : 0);
 }
```

- Thêm 1 phần tử vào đầu danh sách liên kết kép:

```
void addFirst(DList& list, TYPEINFO x){
	NodePtr p;
	p = tao_Node(x);
	if (isEmpty(list)){
		list.first = p;
		list.last = p;
	}
	else{
		p->next = list.first;
		list.first->pre = p;
		list.first = p;
	}
}
```

- Thêm 1 phần tử vào giữa danh sách liên kết kép (Thêm sau phần tử q):

```
void addAfterQ(NODEPTR q, TYPEINFO x)
{
	NODEPTR p;
	p = createNode(x);
	p->next=q->next;
	p->pre = q;
	q->next=p;
	q->next->pre=p;
}
```

- Thêm 1 phần tử vào cuối DSLK kép

```
//them cuoi
void addLast(DList& l, TYPEINFO x)
{
	NodePtr p;
	p = createNode(x);
	if (isEmpty(l)){
		l.first = p;
		l.last = p;
	}
	else{
		l.last->next = p;
		p->pre = l.last;
		l.last = p;
	}
}
```

- Đoạn mã xóa 1 phần tử ở đầu

```
void deleteFirst(DList &l){
	if (isEmpty(l) != NULL){
		l.first = l.first->next;
	}
```

danh sách liên kết kép	}

- Đoạn mã xóa phần tử cuối của danh sách liên kết kép	void deleteLast(DList& l){
	if (l.first != NULL)
	cout << "\n List is empty!";
	else if (l.first == l.last){
	delete l.first;
	l.first = l.last = NULL;
	}
	else{
	NodePtr p = l.last;
	l.last->pre->next = NULL;
	l.last = l.last->pre;
	delete p;
	}
	}

2. Danh sách liên kết vòng

2. **DSLK VÒNG** (circular linked list)

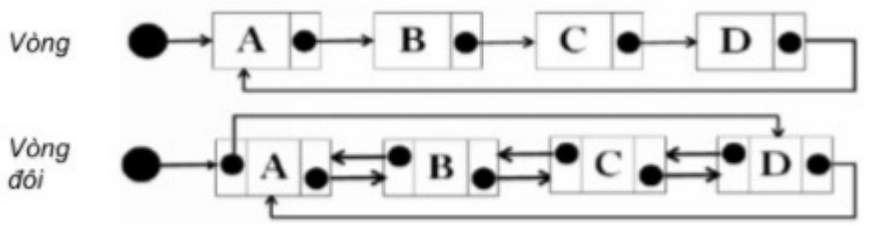
- **Định nghĩa** : Danh sách liên kết vòng (Circular Linked List) là một biến thể của danh sách liên kết (Linked List), trong đó phần tử đầu tiên trỏ tới phần tử cuối cùng và phần tử cuối cùng trỏ tới phần tử đầu tiên.

- Cả hai loại Danh sách liên kết đơn (Singly Linked List) và Danh sách liên kết đôi (kép - Doubly Linked List) đều có thể tạo thành dạng Danh sách liên kết vòng.

- **Cách biểu diễn**: Mỗi phần tử của danh sách liên kết đơn gồm 02 vùng:

- Vùng chứa dữ liệu *data*
- Vùng chứa liên kết đến phần tử kế *next*

- **Hình ảnh minh họa:**



Chúng ta sẽ tìm hiểu danh sách liên kết vòng từ DSLK đơn.

- Khai báo kiểu dữ liệu cho danh sách số nguyên:	`typedef int TYPEINFO;`
	`struct Node{`
	`  TYPEINFO data;`
	`  Node* link;`
	`};`
	`typedef Node* NodePtr;`
	`struct List{`
	`  NodePtr first;`
	`  NodePtr last;`
	`};`

5.3 NỘI DUNG THỰC HÀNH

5.3.1 Danh sách liên kết kép

5.3.1.1 Trên số nguyên

5.3.1.1.1 Câu 1.1.1:

Cho đoạn mã sau, thêm một phần tử vào DSLK kép, yêu cầu sinh viên nhập đoạn mã và biên dịch chương trình.

- Trong file **`thuvien_lab5_dslkkep_songuyen.h`** như sau:

Dòng	Mã lệnh
1	`#include<iostream>`
2	`using namespace std;`
3	`typedef int TYPEINFO;`
4	`struct Node{`
5	`  TYPEINFO data;`
6	`  Node* next;`
7	`  Node* pre;`
8	`};`
9	`typedef Node* Nodeptr;`
10	`struct DList{`
11	`  Nodeptr first;`
12	`  Nodeptr last;`
13	`};`
14	
15	`void init(DList& list);`
16	`int isEmpty(DList list);`
17	`Nodeptr createNode(TYPEINFO x);`
18	`void addFirst(DList& list, TYPEINFO x);`
19	`void createList(DList& list);`
20	`void printList(DList list);`

```
21  void printListReverse(DList list);
```

- Trong file **caidat_lab5_dslkkep_songuyen.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab5_dslkkep_songuyen.h"`
2	`void init(DList& list){`
3	` list.first = NULL;`
4	` list.last = NULL;`
5	`}`
6	`int isEmpty(DList list){`
7	` return (list.first == NULL ? 1 : 0);`
8	`}`
9	`Nodeptr createNode(TYPEINFO x){`
10	` Nodeptr p;`
11	` p = new Node;`
12	` p->data = x;`
13	` p->next = NULL;`
14	` p->pre = NULL;`
15	` return p;`
16	`}`
17	`void addFirst(DList& list, TYPEINFO x){`
18	` Nodeptr p;`
19	` p = createNode(x);`
20	` if (isEmpty(list)){`
21	`  list.first = p;`
22	`  list.last = p;`
23	` }`
24	` else{`
25	`  p->next = list.first;`
26	`  list.first->pre = p;`
27	`  list.first = p;`
28	
29	` }`
30	`}`
31	`void createList(DList& list){`
32	` int x;`
33	` init(list);`
34	` do{`
35	`  cout << "\n Enter x= (-99 to exit)";`
36	`  cin >> x;`
37	`  if (x == -99)`
38	`   break;`
39	`  addFirst(list, x);`

```
40          } while (1);
41      }
42
43      void printList(DList list){
44          Nodeptr p = list.first;
45          while (p != NULL){
46              cout << p->data << " ";
47              p = p->next;
48          }
49      }
```

- Trong file **chuongtrinh_lab5_dslkkep_songuyen.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab5_dslkkep_songuyen.h"`
2	`void main(){`
3	`    DList list;`
4	`    init(list);`
5	`    createList(list);`
6	`    cout << "\n List:";`
7	`    printList(list);`
8	`}`

Yêu cầu:

a/. Nhập đoạn mã trên và chạy chương trình

b/. Viết thêm hàm xuất (**printListReverse**) nghĩa là xuất ra một danh sách chạy theo thứ tự ngược lại với hàm (**printList**)

5.3.1.1.2 Câu 1.1.2:

Đoạn mã sau là hàm thêm một phần tử vào danh sách liên kết kép ở vị trí cuối, yêu cầu sinh viên nhập thêm vào chương trình trên và chạy chương trình.

Dòng	Mã lệnh
1	`//them cuoi`
2	`void addLast(DList& l, TYPEINFO x)`
3	`{`
4	`    Nodeptr p;`
5	`    p = createNode(x);`
6	`    if (isEmpty(l)){`
7	`        l.first = p;`
8	`        l.last = p;`
9	`    }`

```
10          else{
11                l.last->next = p;
12                p->pre = l.last;
13                l.last = p;
14          }
15    }
```

5.3.1.1.3 Câu 1.1.3:

Hãy viết hàm xóa đầu để xóa phần tử đầu danh sách liên kết kép, có nguyên mẫu hàm như sau:

```
void deleteFirst(DList &list)
```

5.3.1.1.4 Câu 1.1.4:

Hãy viết hàm xóa cuối để xóa phần tử cuối danh sách liên kết kép, có nguyên mẫu hàm như sau:

```
void deleteLast(DList &list)
```

5.3.1.1.5 Câu 1.1.5:

Hãy viết hàm tìm kiếm một node có giá trị là x có trong danh sách liên kết kép, nếu có trả về vị trí tìm được, nếu không có xuất ra thông báo không có phần tử cần tìm, có nguyên mẫu hàm như sau:

```
int findXPosition(DList list, TYPEINFO x)
```

5.3.1.2 Trên đối tượng

5.3.1.2.1 Câu 1.2.1:

Cho đoạn mã sau, thêm một phần tử vào DSLK kép, yêu cầu sinh viên nhập đoạn mã và biên dịch chương trình.

- Trong file **thuvien_lab5_dslkkep_doituong.h** như sau:

Dòng	Mã lệnh
1	`#include<iostream>`
2	`using namespace std;`
3	
4	`struct Student{`
5	`    char id[10];`
6	`    char fullName[30];`
7	`    float score;`
8	`};`
9	`struct Node{`

```
10      Student data;
11      Node* next;
12      Node* pre;
13  };
14  typedef Node* Nodeptr;
15  struct DList{
16      Nodeptr first;
17      Nodeptr last;
18  };
19
20  void init(DList& list);
21  int isEmpty(DList list);
22  Nodeptr createNode(Student x);
23  void printStudentInfo(Student x);
24  void addFirst(DList& list, Student x);
25  void createList(DList& list);
26  void printList(DList list);
27  void printListReverse(DList list);
```

- Trong file **caidat_lab5_dslkkep_doituong.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab5_dslkkep_doituong.h"`
2	`void init(DList& list){`
3	`    list.first = NULL;`
4	`    list.last = NULL;`
5	`}`
6	`int isEmpty(DList list){`
7	`    return (list.first == NULL ? 1 : 0);`
8	`}`
9	`Nodeptr createNode(Student x){`
10	`    Nodeptr p;`
11	`    p = new Node;`
12	`    p->data = x;`
13	`    p->next = NULL;`
14	`    p->pre = NULL;`
15	`    return p;`
16	`}`
17	`void addFirst(DList& list, Student x){`
18	`    Nodeptr p;`
19	`    p = createNode(x);`
20	`    if (isEmpty(list)){`
21	`        list.first = p;`
22	`        list.last = p;`

```
23          }
24          else{
25                p->next = list.first;
26                list.first->pre = p;
27                list.first = p;
28
29          }
30    }
31    void createList(DList& list){
32          Student student;
33          init(list);
34          do{
35                cout << "\n Enter Id (enter 0 to exit):";
36                cin.getline(student.id, 10);
37                while (isDuplicate(list, student.id)){
38                      cout << "\n Id is duplicate";
39                      cout << "\n Enter id again,please:";
40                      cin.getline(student.id, 10);
41                }
42                if (strcmp(student.id, "0") == 0)  //NHAP MA SO = 0
43    DE THOAT
44                      break;
45                cout << "\n Enter fullname:";
46                cin.getline(student.fullName, 30);
47                cout << "\n Enter score:";
48                cin >> student.score;
49                cin.ignore();
50                addFirst(list, student);
51          } while (1);
52    }
53    void printStudentInfo(Student x){
54         cout << "\n Id:" <<x.id<<endl;
55         cout << "\n Fullname:" <<x.fullName<<endl;
56         cout << "\n Score:"<<x.score<<endl;
57    }
58
59    void printList(DList list){
60          Nodeptr p = list.first;
61          while (p != NULL){
62                printStudentInfo(p->data);
63                p = p->next;
64          }
65    }
```

- Trong file **chuongtrinh_lab5_dslkkep_doituong.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab5_dslkkep_doituong.h"`
2	`void main(){`
3	`DList list;`
4	`init(list);`
5	`createList(list);`
6	`cout << "\n List:";`
7	`printList(list);`
8	`}`

Yêu cầu:

c/. Nhập đoạn mã trên và chạy chương trình

d/. Viết thêm hàm xuất (**printListReverse**) nghĩa là xuất ra một danh sách chạy theo thứ tự ngược lại với hàm (**printList**)

5.3.1.2.2 Câu 1.2.2:

Đoạn mã sau là hàm thêm một phần tử vào danh sách liên kết kép ở vị trí cuối, yêu cầu sinh viên nhập thêm vào chương trình trên và chạy chương trình.

Dòng	Mã lệnh
1	`//them cuoi`
2	`void addLast(DList& l, Student x)`
3	`{`
4	`Nodeptr p;`
5	`p = createNode(x);`
6	`if (isEmpty(l)){`
7	`l.first = p;`
8	`l.last = p;`
9	`}`
10	`else{`
11	`l.last->next = p;`
12	`p->pre = l.last;`
13	`l.last = p;`
14	`}`
15	`}`

5.3.1.2.3 Câu 1.2.3:

Hãy viết hàm xóa đầu để xóa sinh viên ở đầu danh sách liên kết kép, có nguyên mẫu hàm như sau: **`void deleteFirst(DList &list)`**

5.3.1.2.4 Câu 1.2.4:

Hãy viết hàm xóa cuối để xóa sinh viên cuối danh sách liên kết kép, có nguyên mẫu hàm như sau: **`void deleteLast(DList &list)`**

5.3.2 Danh sách liên kết vòng

5.3.2.1 Trên số nguyên

5.3.2.1.1 Câu 2.1.1:

Sau đây là đoạn mã về danh sách liên kết vòng, yêu cầu sinh viên nhập đoạn mã vào:

- Trong file **`thuvien_lab5_dslkvong_songuyen.h`** như sau:

Dòng	Mã lệnh
1	`#include<iostream>`
2	`using namespace std;`
3	`typedef int TYPEINFO;`
4	`struct Node{`
5	` TYPEINFO data;`
6	` Node* link;`
7	`};`
8	`typedef Node* Nodeptr;`
9	`struct List{`
10	` Nodeptr first;`
11	` Nodeptr last;`
12	`};`
13	`Nodeptr createNode(TYPEINFO x);`
14	`void init(List &list);`
15	`int isEmpty(List list);`
16	`void addFirst(List &list, TYPEINFO x);`
17	`void addLast(List &list, TYPEINFO x);`
18	`void createList(List &list);`
19	`void printList(List list);`
20	`void deleteFirst(List &list);`

- Trong file **`caidat_lab5_dslkvong_songuyen.cpp`** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab5_dslkvong_songuyen.h"`
2	`Nodeptr createNode(TYPEINFO x){`
3	` Nodeptr p = new Node;`
4	` if (p == NULL)`
5	`  return 0;`
6	` p->link = NULL;`

```
7       p->data = x;
8       return p;
9   }
10
11  void init(List& list){
12      list.first = list.last = NULL;
13  }
14  int isEmpty(List list){
15      if (list.first == NULL)
16          return 1;
17      return 0;
18  }
19
20  void addFirst(List& list, TYPEINFO x){
21      Nodeptr p = createNode(x);
22      if (isEmpty(list) == 1)
23          list.first = list.last = p;
24      else{
25          p->link = list.first;
26          list.first = p;
27      }
28      list.last->link = list.first;
29  }
30  void createList(List& list){
31      int x;
32      do{
33          cout << "\n Enter x=(exit: -99)";
34          cin >> x;
35          if (x == -99)
36              break;
37          addFirst(list, x);
38          //addLast(list,x);
39      } while (1);
40  }
41
42  void printList(List list){
43      if (list.first != NULL){
44          cout << "\n All items in list:";
45          Nodeptr p = list.first;
46          do    {
47              cout << p->data << " ";
48              p = p->link;
49          } while (p != list.first);
50      }
51      else
```

```
52              cout << "\n List is empty";
53      }
```

- Trong file **chuongtrinh_lab5_dslkvong_songuyen.cpp** như sau:

Dòng	Mã lệnh
1	#include"thuvien_lab5_dslkvong_songuyen.h"
2	void main(){
3	List list;
4	init(list);
5	createList(list);
6	printList(list);
7	system("pause");
8	}

Yêu cầu:

a/. Sinh viên hãy nhập đoạn mã trên và chạy chương trình.

b/. Sinh viên hãy viết thêm hàm thêm cuối danh sách, sau đó gọi và chạy thử chương trình

5.3.2.1.2 Câu 2.1.2:

Cho hàm xóa đầu của DSLK vòng như sau:

Dòng	Mã lệnh
1	void deleteFirst(List& list){
2	if (list.first != NULL){
3	if (list.first != list.last){
4	Nodeptr p = list.first;
5	list.first = list.first->link;
6	list.last->link = list.first;
7	delete p;
8	}
9	else
10	list.first = NULL;
11	}
12	}

Yêu cầu:

a/. Sinh viên hãy nhập đoạn mã trên vào, gọi và chạy thử chương trình.

b/. Sinh viên viết thêm hàm xóa cuối và chạy thử chương trình.

5.3.2.1.3 Câu 2.1.3:

Sinh viên hãy viết hàm tìm kiếm node có giá trị bằng x có trong DSLK vòng.

5.3.3 Trên đối tượng

5.3.3.1 Câu 2.2.1:

- Trong file **thuvien_lab5_dslkvong_doituong.h** như sau:

Dòng	Mã lệnh
1	`#include<iostream>`
2	`using namespace std;`
3	`struct Student{`
4	` char id[10];`
5	` char fullName[30];`
6	` float score;`
7	`};`
8	
9	`struct Node{`
10	` Student data;`
11	` Node* link;`
12	`};`
13	`typedef Node* Nodeptr;`
14	`struct List{`
15	` Nodeptr first;`
16	` Nodeptr last;`
17	`};`
18	`Nodeptr createNode(Student x);`
19	`void init(List &list);`
20	`int isEmpty(List list);`
21	`void addFirst(List &list, Student x);`
22	`void addLast(List &list, Student x);`
23	`void createList(List &list);`
24	`void printList(List list);`
25	`void deleteFirst(List &list);`

- Trong file **caidat_lab5_dslkvong_doituong.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab5_dslkvong_doituong.h"`
2	`Nodeptr createNode(Student x){`
3	` Nodeptr p = new Node;`
4	` if (p == NULL)`

```
5               return 0;
6         p->link = NULL;
7         p->data = x;
8         return p;
9   }
10
11  void init(List& list){
12        list.first = list.last = NULL;
13  }
14  int isEmpty(List list){
15        if (list.first == NULL)
16              return 1;
17        return 0;
18  }
19
20  void addFirst(List& list, Student x){
21        Nodeptr p = createNode(x);
22        if (isEmpty(list) == 1)
23              list.first = list.last = p;
24        else{
25              p->link = list.first;
26              list.first = p;
27        }
28        list.last->link = list.first;
29  }
30  void createList(List& list){
31        Student student;
32        init(list);
33        do{
34              cout << "\n Enter id (enter 0 to exit):";
35              cin.getline(student.id, 10);
36              while (isDuplicate(list, student.id)){
37                    cout << "\n Id is duplicate";
38                    cout << "\n Enter id again,please:";
39                    cin.getline(student.id, 10);
40              }
41              if (strcmp(student.id, "0") == 0)  //NHAP MA SO = 0
42  DE THOAT
43                    break;
44              cout << "\n Enter fullname:";
45              cin.getline(student.fullName, 30);
46              cout << "\n Enter score:";
47              cin >> student.score;
48              cin.ignore();
49              addFirst(list, x);
```

```
50              //addLast(list,x);
51        } while (1);
52  }
53
54  void printList(List list){
55        if (list.first != NULL){
56              cout << "\n All items in list:";
57              Nodeptr p = list.first;
58              do    {
59                    printStudentInfo(p->data);
60                    p = p->link;
61              } while (p != list.first);
62        }
63        else
64              cout << "\n List is empty";
65  }
```

- Trong file **chuongtrinh_lab5_dslkvong_songuyen.cpp** như sau:

Dòng	Mã lệnh
1	#include"thuvien_lab5_dslkvong_songuyen.h"
2	void main(){
3	List list;
4	init(list);
5	createList(list);
6	printList(list);
7	system("pause");
8	}

Yêu cầu:

c/. Sinh viên hãy nhập đoạn mã trên và chạy chương trình.

d/. Sinh viên hãy viết thêm hàm thêm cuối danh sách, sau đó gọi và chạy thử chương trình

5.3.3.1.1 Câu 2.2.2:

Cho hàm xóa đầu của DSLK vòng như sau:

Dòng	Mã lệnh
1	void deleteFirst(List& list){
2	if (list.first != NULL){
3	if (list.first != list.last){
4	Nodeptr p = list.first;
5	list.first = list.first->link;

```
6                       list.last->link = list.first;
7                       delete p;
8                 }
9                 else
10                      list.first = NULL;
11          }
12    }
```

Yêu cầu:

c/. Sinh viên hãy nhập đoạn mã trên vào, gọi và chạy thử chương trình.

d/. Sinh viên viết thêm hàm xóa cuối và chạy thử chương trình.

5.4 BÀI TẬP VỀ NHÀ

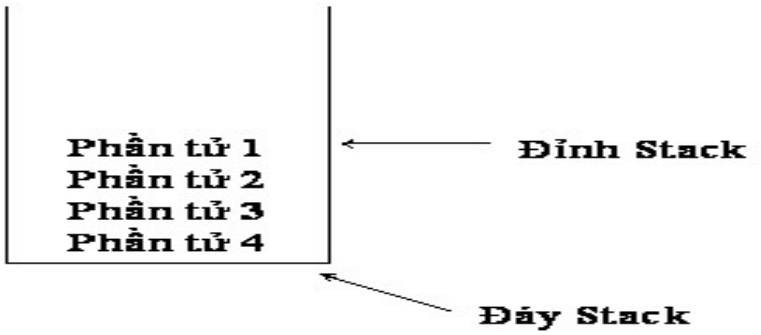
6 DANH SÁCH ĐẶC BIỆT (STACK & QUEUE)

6.1 PHẦN 1: STACK (NGĂN XẾP)

6.1.1 TÓM TẮT LÝ THUYẾT

1. Stack (ngăn xếp)

- **Định nghĩa** : Một ngăn xếp (stack) là một cấu trúc dữ liệu mà việc thêm vào và loại bỏ được thực hiện tại một đầu (gọi là đỉnh – top của stack).
- Một ngăn xếp đĩa đặt trên bàn cho ta hình ảnh trực quan của ngăn xếp, muốn thêm vào ngăn xếp đó 1 đĩa ta để đĩa mới trên đỉnh ngăn xếp, muốn lấy các đĩa ra khỏi ngăn xếp ta cũng phải lấy đĩa trên trước.
- Như vậy ngăn xếp là một cấu trúc có tính chất "vào sau - ra trước" hay LIFO (last in - first out).
- **Cách biểu diễn:**



Nội dung	Stack – cài đặt danh sách đặc	Stack – Cài đặt danh sách liên kết

- Khai báo cấu trúc Stack cho danh sách số nguyên:	`#define MAXSTACK 100`
	`typedef int TYPE;`
	`struct Stack{`
	`  int sp;`
	`TYPE elem[MAXSTACK];`
	`};`
		`typedef int TYPEINFO;`
		`struct Node{`
		`  TYPEINFO data;`
		`  Node* link;`
		`};`
		`typedef Node* NodePtr;`
		`struct Stack{`
		`  NodePtr top;`
		`}`
		`;`

- Khởi tạo stack	`void init(Stack &s){ s.sp=-1; }`	`void init(Stack& s){ s.top=NULL; }`
- Kiểm tra danh sách rỗng hay không	`int isEmpty(Stack s){ return s.sp==-1; }`	`int isEmpty(Stack s){ return s.top==NULL; }`
- Kiểm tra Stack có đầy hay chưa	`int isFull(Stack s){ return s.sp==MAXSTACK-1; }`	
- Tạo node	`NodePtr createNode(TYPEINFO x){ NodePtr p; p = new Node; p->data=x; p->link=NULL; return p; }`	

- **Có 3 thủ tục chính trên Stack**
  - **push:** Thêm một phần tử vào ngăn xếp
  - **pop**: Lấy ra một phần tử khỏi ngăn xếp
  - **top:** Xem phần tử đầu ngăn xếp

6.1.2 NỘI DUNG THỰC HÀNH

6.1.2.1 Cài đặt ngăn xếp bằng danh sách liên kết

6.1.2.1.1 Câu 1.2.1:

Sinh viên nhập đoạn mã sau và chạy chương trình

- Trong file **`thuvien_lab6_stack_dslk.h`** có đoạn mã như sau, hãy nhập vào bài của bạn:

Dòng	Mã lệnh

```
1   #include<iostream>
2   using namespace std;
3   typedef int TYPEINFO;
4   struct Node{
5         TYPEINFO data;
6         Node* link;
7   };
8   typedef Node* NodePtr;
9   struct Stack{
10        NodePtr top;
11  };
12  void init(Stack& s);
13  int isEmpty(Stack s);
14  NodePtr createNode(TYPEINFO x);
15  void push(Stack &s, TYPEINFO a);
16  int pop(Stack &s);
17  int top(Stack &s);
18  void inputStack(Stack &s);
19  void print(Stack s);
```

- Trong file **caidat_lab6_stack_dslk.cpp** có đoạn mã như sau, hãy nhập vào bài của bạn:

Dòng	Mã lệnh
1	`#include"thuvien_lab6_stack_dslk.h"`
2	`const int STOP = -99;`
3	`void init (Stack &s){`
4	`  s.top=NULL;`
5	`}`
6	`int isEmpty(Stck s){`
7	`  return s.top==NULL;`
8	`}`
9	`NodePtr createNode(TYPEINFO x){`
10	`  NodePtr p;`
11	`  p = new Node;`
12	`  p->data=x;`
13	`  p->link=NULL;`
14	`  return p;`
15	`}`
16	`void push(Stack &s, TYPEINFO a){`
17	`  NodePtr p;`
18	`  p=createNode(a);`
19	`  if(isEmpty(s))`
20	`  s.top =p;`

```
21        else{
22              p->link=s.top;
23              s.top=p;
24        }
25  }
26  void inputStack(Stack& s){
27        int x;
28        while(1){
29              cout<<"\n Enter data (enter -99 to exit):";
30              cin>>x;
31              if(x==STOP)
32                    break;
33              push(s,x);
34        }
35  }
36  void print(Stack s){
37        NodePtr p=s.top;
38        while(p!=NULL){
39              cout<<p->data<<" ";
40              p=p->link;
41  }
42
```

- Trong file **chuongtrinh_lab6_stack_dslk.cpp** có đoạn mã như sau, hãy nhập vào bài của bạn:

Dòng	Mã lệnh
1	`#include"thuvien_lab6_stack_dslk.h"`
2	`void main(){`
3	`  Stack s;`
4	`  init(s);`
5	`  inputStack(s);`
6	`  print(s);`
7	`}`

Yêu cầu:

- Nhập đoạn mã trên và chạy chương trình

6.1.2.1.2 Câu 1.2.2:

Cho đoạn mã, lấy ra một phần tử khỏi ngăn xếp như sau:

Dòng	Mã lệnh

```
1   int pop(Stack& s)
2   {
3         NodePtr p;
4         TYPEINFO a;
5         if (!isEmpty(s))
6         {
7               p = s.top;
8               s.top = p->link;
9               a = p->data;
10              delete p;
11              return a;
12        }
13  }
```

Yêu cầu:

a/. Nhập đoạn mã trên vào chương trình

b/. Gọi trong **`void main`**, xuất ra thông tin của phần tử vừa lấy ra khỏi ngăn xếp, và xuất ra danh sách các phần tử còn lại có trong ngăn xếp.

6.1.2.1.3 Câu 1.2.3:

Sinh viên tự viết hàm **`int top(Stack &s)`** để xem thông tin phần tử đầu stack.

6.2 PHẦN 2: QUEUE (HÀNG ĐỢI)

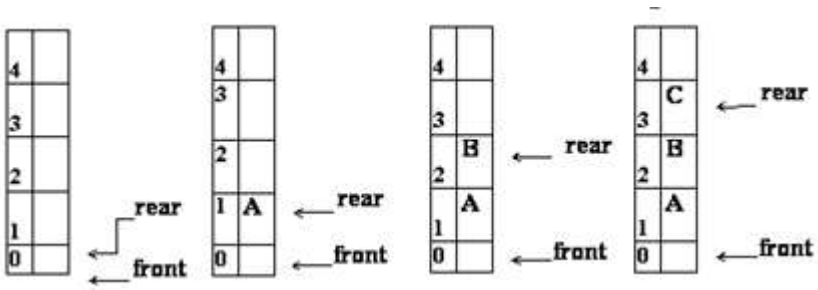
6.2.1 MỤC TIÊU:

- Khai báo queue dùng danh sách đặc
- Khai báo queue dùng danh sách liên kết
- Cài đặt các thao tác trên queue bằng danh sách liên kết

6.2.2 TÓM TẮT LÝ THUYẾT

1. Queue

- Một hàng (queue) là một cấu trúc dữ liệu mà việc thêm vào được thực hiện ở một đầu (rear) và việc lấy ra được thực hiện ở đầu còn lại (front).
- Queue làm việc theo cơ chế: Phần tử vào trước sẽ ra trước – FIFO (First In First Out)
  - **Hình ảnh minh họa:**
    - Biến đầu front chỉ ra vị trí thực hiện phép loại bỏ
    - Biến cuối rear để chỉ ra vị trí thực hiện phép thêm vào.



Nội dung	queue – cài đặt DS Đặc	queue – Cài đặt DSLK

- Khai báo cấu trúc queue cho danh sách số nguyên:	`struct Queue{`
	`  int front, rear;`
	`  int elem[MAX];`
	`};`
		`typedef struct Node *NodePtr;`
		`typedef int TYPEINFO;`
		`struct Node{`
		`  int data;`
		`  Node* link;`
		`};`
		`struct Queue`
		`{`
		`  NodePtr front, rear;`
		`};`

<table>
<tr><td></td><td></td><td></td></tr>
<tr><td>- Khởi tạo queue</td><td>

```
void init(Queue &q)
{
      q.front = -1;
      q.rear = -1;
}
```

</td><td>

```
void init(Queue &q)
{
      q.front = NULL;
      q.rear = NULL;
 }
```

</td></tr>
<tr><td>- Kiểm tra danh sách rỗng hay không</td><td>

```
int isEmpty(Queue q)
{
      if (q.front == -1)
            return 1;
      return 0;
 }
```

</td><td>

```
int isEmpty(Queue q)
{
      if (q.front == NULL)
            return 1;
      return 0;
 }
```

</td></tr>
<tr><td>- Kiểm tra Queue có đầy hay chưa</td><td colspan="2">

```
int isFull(Queue q)
{
      return (q.rear - q.front == MAX - 1);
}
```

</td></tr>
<tr><td>- Tạo node</td><td colspan="2">

```
NodePtr createNode(TYPEINFO x)
{
      NodePtr p;
      p = new Node;
      p->data = x;
      p->link = NULL;
      return p;
}
```

</td></tr>
</table>

- Có 3 thủ tục chính trên Queue

- **enQueue:** Thêm một phần tử vào Queue
- **deQueue**: Lấy ra một phần tử khỏi Queue
- **getFront:** Xem phần tử đầu Queue

6.2.3 NỘI DUNG THỰC HÀNH

6.2.3.1 Cài đặt Queue bằng danh sách liên kết

6.2.3.1.1 Câu 2.3.1:

Cho đoạn mã sau:

- Trong file **thuvien_lab6_queue_dslk.h** có đoạn mã:

Dòng	Mã lệnh
1	`#include<iostream>`
2	`using namespace std;`
3	`typedef struct Node* NodePtr;`
4	`typedef int TYPEINFO;`
5	`struct Node{`
6	` int data;`
7	` Node* link;`
8	`};`
9	`struct Queue{`
10	` NodePtr front, rear;`
11	`};`
12	`void init(Queue& q);`
13	`int isEmpty(Queue q);`
14	`NodePtr createNode(TYPEINFO x);`
15	`void enQueue(Queue& q, TYPEINFO x);`
16	`void setValue(Queue& q);`
17	`void print(Queue q);`

- Trong file **caidat_lab6_queue_dslk.cpp** có đoạn mã như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab6_queue_dslk.h"`
2	`const int STOP = -99;`
3	`void init(Queue &q){`
4	` q.front = NULL;`
5	` q.rear = NULL;`
6	`}`
7	`int isEmpty(Queue q){`

```
8       if (q.front == NULL)
9             return 1;
10      return 0;
11    }
12    NodePtr createNode(TYPEINFO x){
13      NodePtr p;
14      p = new Node;
15      p->data = x;
16      p->link = NULL;
17      return p;
18    }
19    void enQueue(Queue &q, TYPEINFO x){
20      NodePtr p;
21      p = createNode(x);
22      if (isEmpty(q)){
23            q.front = p;
24            q.rear = p;
25      }
26      else{
27            q.rear->link = p;
28            q.rear = p;
29      }
30    }
31    void setValue(Queue &q){
32      int x;
33      while (1){
34            cout << "\n Enter data (enter -99 o exit):";
35            cin >> x;
36            if (x == STOP) break;
37            enQueue(q, x);
38      }
39    }
40
41    void print(Queue q){
42      NodePtr p;
43      p = q.front;
44      while (p != NULL){
45            cout << p->data << " ";
46            p = p->link;
47      }
48    }
49
```

- Trong file **chuongtrinh_lab6_queue_dslk.cpp** có đoạn mã như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab6_queue_dslk.h"`
2	`void main(){`
3	`Queue q;`
4	`init(q);`
5	`setValue(q);`
6	`print(q);`
7	`system("pause");`
8	`}`

- Các bạn nhập đoạn mã trên và chạy chương trình cài đặt queue bằng DSLK.

6.2.3.1.2 Câu 2.3.2:

Sau đây là hàm deQueue, lấy một phần tử ra khỏi hàng đợi, sinh viên hãy nhập vào và gọi hàm trong **`void main`**.

Dòng	Mã lệnh
1	`TYPEINFO deQueue(Queue &q){`
2	`if (isEmpty(q))`
3	`return -1;`
4	`else{`
5	`NodePtr p = q.front;`
6	`q.front = p->link;`
7	`TYPEINFO t = p->data;`
8	`delete p;`
9	`return t;`
10	`}`
11	`}`

6.2.3.1.3 Câu 2.3.3:

Sinh viên tự viết hàm **`TYPEINFO getFront(Queue q)`** và gọi hàm trong void main, để xem phần tử đầu hàng đợi.

7 CÂY NHỊ PHÂN - CÂY NHỊ PHÂN TÌM KIẾM

7.1 MỤC TIÊU:

- Khai báo cây nhị phân
- Cách cài đặt một cây nhị phân
- Khai báo cây nhị phân tìm kiếm BST
- Cài đặt cây nhị phân tìm kiếm BST
- Viết chương trình thêm nút, xóa nút trên cây BST

7.2 TÓM TẮT LÝ THUYẾT:

1. **Cây nhị phân** *(binary tree)*

- Là cây có bậc 2

Bậc:

- *Bậc của cây* : số cây con tối đa của một nút trên cây. Cây nhị phân có bậc là 2.
- *Bậc của nút* : số nút con của nút đó

Mức:

- Node gốc ở mức 0.
- Node gốc của các cây con của một node ở mức *m* là *m+1*.

Chiều cao:

- Cây rỗng có chiều cao là 0.
- Chiều cao của cây là chiều cao lớn nhất của một trong hai cây con cộng thêm một
- (Hoặc: mức lớn nhất của các node cộng 1)

Đường đi *(path)*

- Tên các node có được trong quá trình đi từ node gốc theo các cây con đến một node nào đó.

Node trước, sau, cha, con:

- Node x là trước node y (node y là sau node x), nếu trên đường đi đến y có x.
- Node x là cha node y (node y là con node x), nếu trên đường đi đến y node x nằm ngay trước node y.

Node lá, trung gian:

- Node lá là node không có cây con nào. Bậc của nút lá là 0.

- Node trung gian không là node gốc hay node lá. Bậc của nút trung gian là khác 0.

Phép duyệt cây

- Duyệt qua từng node của cây (mỗi node chỉ được duyệt qua một lần)

Cách duyệt:

- Gọi: N(node), L(left), R(right)
- Chính thức: NLR, LNR, LRN, NRL, RNL, RLN
- Chuẩn: NLR (preorder), LNR (inorder), LRN (postorder)

2. Cây nhị phân tìm kiếm (Binary Search Tree – BST)

- Một cây nhị phân tìm kiếm (BST) là một cây nhị phân rỗng hoặc mỗi node của cây này có các đặc tính sau:
  - Khóa của node gốc lớn hơn khóa của tất cả các node của cây con bên trái và nhỏ hơn khóa của tất cả các node của cây con bên phải.
  - Các cây con (bên trái, phải) là BST
- **Các tính chất khác của BST**
  - Node cực trái (hay cực phải):
    - Xuất phát từ node gốc
    - Đi sang trái (hay phải) đến khi không đi được nữa
  - Khóa của node cực trái là nhỏ nhất trong BST
  - Khóa của node cực phải là lớn nhất trong BST
  - BST là cây nhị phân có tính chất:
    - Khóa của node gốc lớn hơn khóa của node cực trái
    - Khóa của node gốc nhỏ hơn khóa của node cực phải

Nội dung	Mã lệnh

- Khai báo cấu trúc Cây nhị phân tìm kiếm cho danh sách số nguyên:	`typedef int TYPEINFO;`
	`struct Node{`
	`  TYPEINFO data;`
	`  Node* left;`
	`  Node* right;`
	`};`
	`typedef Node* NodePtr;`
	`typedef NodePtr BST;`

- Khởi tạo Cây	`void init(BST &root){`
	`  root = NULL;`
	`}`

- Tạo node	`NodePtr createNode(int x){`
	`  NodePtr p;`
	`  p = new Node;`
	`  p->data = x;`
	`  p->left = NULL;`
	`  p->right = NULL;`
	`  return p;`
	`}`

- Thêm phần tử vào cây nhị phân tìm kiếm	`void insertTree(BST &root, int x){`
	`  if (root == NULL)`
	`  root = createNode(x);`
	`  else{`
	`  if (x == root->data){`
	`  cout << "\n Node already exist";`
	`  return;`
	`  }`
	`  else`
	`  if (x<root->data)`
	`  insertTree(root->left, x);`
	`  else`
	`  insertTree(root->right, x);`
	`  }`
	`}`

- Tìm node a có tồn tại trong cây BST	`NodePtr findNode(BST tree, TYPEINFO a){`
	`  NodePtr p = tree;`
	`  while (p != NULL){`
	`  if (p->data == a)`
	`  return p;`
	`  else{`
	`  if (p->data>a) p = p->left;`
	`  else`
	`  p = p->right;`
	`  }`
	`  }`
	`  return p;`
	`}`

7.3 NỘI DUNG THỰC HÀNH

7.3.1 Câu 1:

Sinh viên nhập đoạn mã sau và chạy chương trình

- Trong file **thuvien_lab7.h** có đoạn mã như sau, hãy nhập vào bài của bạn:

Dòng	Mã lệnh
1	`#include <iostream>`
2	`#include <windows.h>`
3	`using namespace std;`
4	`typedef int TYPEINFO;`
5	`struct Node{`
6	` TYPEINFO data;`
7	` Node* left;`
8	` Node* right;`
9	`};`
10	`typedef Node* NodePtr;`
11	`typedef NodePtr BST;`
12	`NodePtr createNode(int x);`
13	`void init(BST &root);`
14	`void gotoxy(short x, short y);`
15	`void insertTree(BST &root, int x);`
16	`void createTree(BST &root);`
17	`void NLR(BST T, int x, int y, int d);`
18	`int sumAllNode(BST t);`
19	`int sumLeafNode(BST t);`
20	`int height(BST tree);`
21	`void traverseNLR(BST &root);`
22	`void traverseLNR(BST &root);`
23	`void traverseLRN(BST &root);`
24	`int calculateLevel(BST root, TYPEINFO x);`
25	`NodePtr findNode(BST tree, TYPEINFO a);`

- Trong file **caidat_lab7.cpp** có đoạn mã như sau, hãy nhập vào bài của bạn:

Dòng	Mã lệnh
1	`#include"thuvien_lab7.h"`
2	`NodePtr createNode(int x){`
3	` NodePtr p;`
4	` p = new Node;`
5	` p->data = x;`
6	` p->left = NULL;`
7	` p->right = NULL;`
8	` return p;`
9	`}`
10	`void init(BST &root){`
11	` root = NULL;`

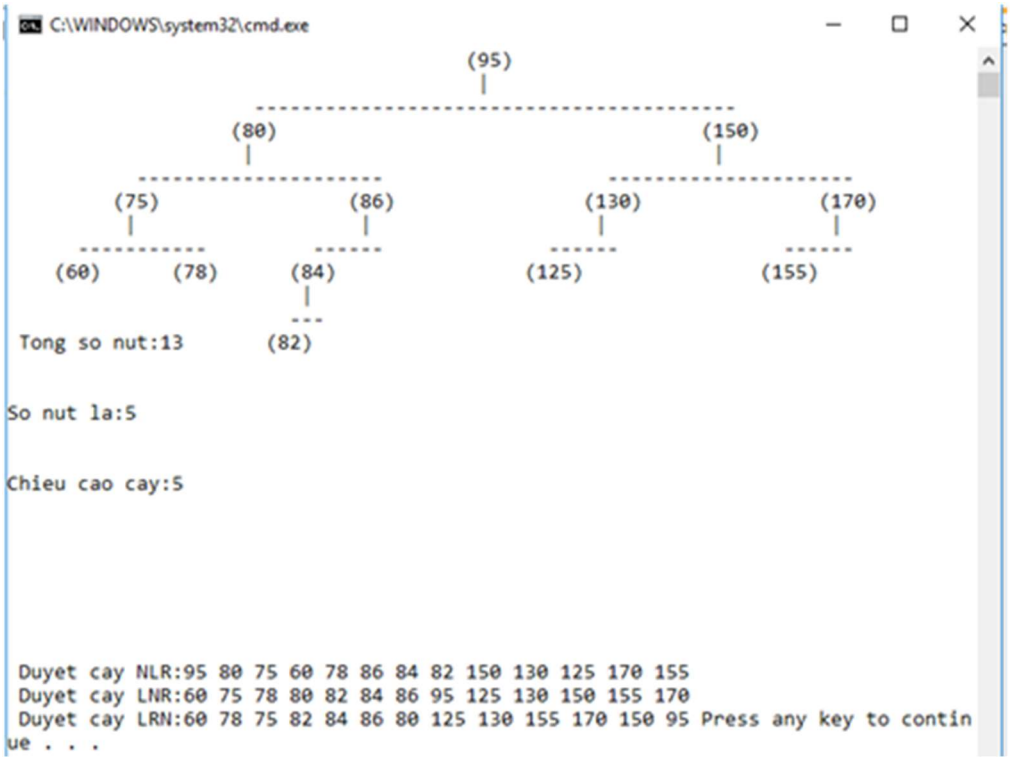
```
12    }
13    void gotoxy(short x, short y){
14          HANDLE hCon = GetStdHandle(STD_OUTPUT_HANDLE);
15          COORD pos;
16          pos.X = x - 1;
17          pos.Y = y - 1;
18          SetConsoleCursorPosition(hCon, pos);
19    }
20    void insertTree(BST &root, int x){
21          if (root == NULL)
22                root = createNode(x);
23          else{
24                if (x == root->data){
25                      cout << "\n Node already exist";
26                      return;
27                }
28                else
29                      if (x<root->data)
30                            insertTree(root->left, x);
31                      else
32                            insertTree(root->right, x);
33          }
34    }
35    void createTree(BST &root){
36          int x;
37          while (1){
38                cout << "\n Enter data (enter -99 to exit):";
39                cin >> x;
40                if (x == -99)
41                      break;
42                insertTree(root, x);
43          }
44    }
45    void NLR(BST T, int x, int y, int d){
46          if (T != NULL){
47                gotoxy(x, y);
48                cout << "(" << T->data << ")";
49
50                if (T->left != NULL){
51                      gotoxy(x + 1, y + 1);
52                      cout << '|';
53                      for (int i = x - d / 2 + 2; i <= x + 2; i++)
54          {
55                            gotoxy(i, y + 2);
56                            cout << '-';
```

```
57                      }
58                      NLR(T->left, x - d / 2, y + 3, d / 2);
59                }
60                if (T->right != NULL){
61                      gotoxy(x + 1, y + 1);
62                      cout << '|';
63                      for (int i = x + 2; i <= x + d / 2 + 2; i++){
64                            gotoxy(i, y + 2);
65                            cout << '-';
66                      }
67                      NLR(T->right, x + d / 2, y + 3, d / 2);
68                }
69          }
}
```

- Trong file **chuongtrinh_lab7.cpp** có đoạn mã như sau, hãy nhập vào bài của bạn:

Dòng	Mã lệnh
1	`#include"thuvien_lab7.h"`
2	`void main(){`
3	`BST tree;`
4	`init(tree);`
5	`createTree(tree);`
6	`system("cls");`
7	
8	`cout << " \n\n\n\n\n\n\n\n\n\n\n\n\n\n";`
9	`cout << " \n\n\n\n\n\n\n\n\n\n\n\n\nCAY BAN VUA TAO LA:";`
10	`cout << "\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n";`
11	`NLR(tree, 40, 1, 40);`
12	`}`

- Khi ta nhập vào dãy số sau: 95, 80, 150, 170, 130, 86, 75, 78, 84, 95, 82, 60, 125, 155. Kết quả xuất cây nhị phân tìm kiếm như sau:



7.3.2 Câu 2:

Các bạn viết thêm các hàm tính tổng số nút có trên cây, số nút lá, và xuất ra chiều cao cây như kết quả trên.

7.3.3 Câu 3:

Viết 3 hàm duyệt cây BST: NLR, LNR, LRN

7.3.4 Câu 4:

Viết hàm kiểm tra xem node có giá trị là x có tồn tại trên cây nhị phân hay không, nếu có trả về vị trí của node đó, nếu không trả về NULL.

7.3.5 Câu 5:

Viết hàm xóa một node trên cây nhị phân tìm kiếm.

8 Cây nhị phân tìm kiếm

9 THUẬT TOÁN SẮP XẾP

9.1 MỤC TIÊU:

Hiểu và cài đặt được các thuật toán sắp xếp cơ bản sau:

- Insertion sort
- Selection sort
- Buble sort
- Quick sort
- Heap sort

9.2 TÓM TẮT LÝ THUYẾT

Thuật toán Insertion sort

Ý tưởng:

Giả sử có một dãy $a_0 , a_1 ,... ,a_{n-1}$ trong đó i phần tử đầu tiên $a_0 , a_1 ,... ,a_{i-1}$ đã có thứ tự.

Tìm cách chèn phần tử a_i vào vị trí thích hợp của đoạn đã được sắp để có dãy mới $a_0 , a_1,... ,a_i$ trở nên có thứ tự. Vị trí này chính là vị trí giữa hai phần tử a_{k-1} và a_k thỏa $a_{k-1} < a_i < a_k$ $(1 \le k \le i)$.

Cài đặt:

```
void insertionSort(int a[], int n )
{       int viTri, i;
        int x;//lưu giá trị a[i] tránh bị ghi đè khi dời chỗ các phần tử.
        for(i=1 ; i<n ; i++) //đoạn a[0] đã sắp
        {
                x = a[i]; viTri = i-1;
            // tìm vị trí chèn x
                while((viTri >= 0)&&(a[viTri] > x))
            {//kết hợp dời chỗ các phần tử sẽ đứng sau x trong dãy mới
                    a[viTri+1] = a[viTri];
                viTri--;
            }
            a[viTri+1] = x; // chèn x vào dãy
        }
}
```

Thuật toán Selection sort

Ý tưởng:

- Chọn phần tử nhỏ nhất trong N phần tử trong dãy hiện hành ban đầu.
- Đưa phần tử này về vị trí đầu dãy hiện hành

- Xem dãy hiện hành chỉ còn N-1 phần tử của dãy hiện hành ban đầu
  - Bắt đầu từ vị trí thứ 2;
  - Lặp lại quá trình trên cho dãy hiện hành... đến khi dãy hiện hành chỉ còn 1 phần tử

Cài đặt:

```
void selectionSort(int a[],int n )
{
        int     min,i,j; // chỉ số phần tử nhỏ nhất trong dãy hiện hành
        for (i=0; i<n-1 ; i++) //chỉ số đầu tiên của dãy hiện hành
        {
        min = i;
                for(j = i+1; j <N ; j++)
                        if (a[j ] < a[min])
                                min = j; // lưu vtrí phần tử hiện nhỏ nhất
                Swap(a[min],a[i]);
    }
}
```

Thuật toán Bubble sort

Ý tưởng:

Xuất phát từ phần tử cuối danh sách ta tiến hành so sánh với phần tử bên trái của nó. Nếu phần tử đang xét có khóa nhỏ hơn phần tử bên trái của nó ta tiến đưa nó về bên trái của dãy bằng cách hoán vị với phần tử bên trái của nó.

Tiếp tục thực hiện như thế đối với bài toán có n phần tử thì sau n –1 bước ta thu được danh sách tăng dần.

Cài đặt:

```
void bubbleSort(int a[],int n)
{
    int     i, j;
    for (i = 0 ; i<n-1 ; i++)
            for (j =n-1; j >i ; j --)
                    if(a[j]< a[j-1])// nếu sai vị trí thì đổi chỗ
                            Swap(a[j], a[j-1]);
}
```

Thuật toán Quick sort:

Ý tưởng:

- Giải thuật QuickSort sắp xếp dãy $a_1, a_2 ..., a_N$ dựa trên việc phân hoạch dãy ban đầu thành 3 phần :
  - Phần 1: Gồm các phần tử có giá trị bé hơn **x**
  - Phần 2: Gồm các phần tử có giá trị bằng **x**
  - Phần 3: Gồm các phần tử có giá trị lớn hơn **x**

  với x là giá trị của một phần tử tùy ý trong dãy ban đầu.
- Sau khi thực hiện phân hoạch, dãy ban đầu được phân thành 3 đoạn:
  - 1. ak $\leq$ x , với k = 1 .. j
  - 2. ak = x , với k = j+1 .. i-1
  - 3. ak $\geq$ x , với k = i..N
- Đoạn 2 đã được sắp
- Đoạn 1 và 3 nếu có hơn 1 phần tử thì tiến hành sắp xếp trên đoạn 1 và 3
- Dãy chỉ được sắp khi cả 3 đoạn 1,2,3 đã được sắp theo thứ tự
- Để sắp xếp các đoạn 1 và 3, ta lần lượt tiến hành việc phân hoạch từng dãy con theo cách như trên. (tiến hành phân hoạch đoạn đó thành 3 đoạn con)

Cài đặt:

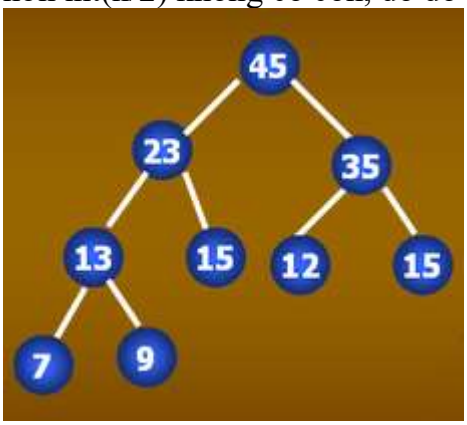
```
void quickSort(int a[], int left, int right)
{       int     i, j, x;
        x = a[(left+right)/2];
        i = left; j = right;
 while(i < j)
        {
        while(a[i] < x) i++;
        while(a[j] > x) j--;
        if(i <= j)
                {
                        swap(a[i],a[j]);
                i++ ; j--;
                }
        }
        if(left<j)
                quickSort(a, left, j);
        if(i<right)
                quickSort(a, i, right);
}
```

Thuật toán Heap sort

Ý tưởng:

Sắp xếp vun đống (Heapsort) dựa trên một cấu trúc dữ liệu được gọi là đống nhị phân (binary heap), gọi đơn giản là đống (heap)

Heap:
Mỗi mảng a[1..n] có thể xem như một cây nhị phân gần đầy (có trọng số là các giá trị của mảng), với gốc ở phần tử thứ nhất, thì hai con là a[2*i+1] và a[2*i+2]) (nếu 2*i<=n hoặc 2*i+1<=n, khi đó các phần tử có chỉ số lớn hơn int(n/2) không có con, do đó là lá) (leaf).



Ví dụ mảng (45,23,35,13,15,12,15,7,9) là một heap

- Một cây nhị phân, được gọi là đống cực đại (max heap) nếu khóa của mọi nút không nhỏ hơn khóa các con của nó. Khi biểu diễn một mảng a[] bởi một cây nhi phân theo thứ tự tự nhiên điều đó nghĩa là a[i]>=a[2*i] và a[i]>=a[2*i+1] với mọi i =1..int(n/2). Ta cũng sẽ gọi mảng như vậy là đống (heap).

Như vậy trong đống a[0] (ứng với gốc của cây) là phần tử lớn nhất.
Mảng bất kỳ chỉ có một phần tử luôn luôn là một đống (heap).

- Một đống cực tiểu (min heap) được định nghĩa theo các bất đẳng thức ngược lại: a[i]<=a[2*i] và a[i]<=a[2*i+1]. Phần tử đứng ở gốc cây cực tiểu là phần tử nhỏ nhất.

Vun đống
Việc sắp xếp lại các phần tử của một mảng ban đầu sao cho nó trở thành đống được gọi là vun đống.
Vun đống tại đỉnh thứ i
Nếu hai cây con gốc 2*i và 2∗i+1 đã là đống thì để cây con gốc i trở thành đống chỉ việc so sánh giá trị a[i] với giá trị lớn hơn trong hai giá trị a[2*i] và a[2*i+1], nếu a[i] nhỏ hơn thì đổi chỗ chúng cho nhau. Nếu đổi chỗ cho a[2∗i], tiếp tục so sánh với con lớn hơn trong hai con của nó cho đên khi hoặc gặp đỉnh lá.
Vun một mảng thành đống
Để vun mảng a[1..n] thành đống ta vun từ dưới lên, bắt đầu từ phần tử a[j]với j =Int(n/2) ngược lên tới a[1]
Sắp xếp bằng vun đống
Đổi chỗ (Swap): Sau khi mảng a[1..n] đã là đống, lấy phần tử a[1] trên đỉnh của đống ra khỏi đống đặt vào vị trí cuối cùng n, và chuyển phần tử thứ cuối cùng a[n] lên đỉnh đống thì phần tử a[n] đã được đứng đúng vị trí.

Vun lại: Phần còn lại của mảng a[1..n-1] chỉ khác cấu trúc đống ở phần tử a[1]. Vun lại mảng này thành đống với n-1 phần tử.
Lặp: Tiếp tục với mảng a[1..n-1]. Quá trình dừng lại khi đống chỉ còn lại một phần tử.

Cài đặt:

```
// n là kích thước của đống
void heapify(int arr[], int n, int i) {
  int largest = i; // khởi tạo largest là gốc
  int left = 2 * i + 1; // left = 2*i + 1
  int right = 2 * i + 2; // right = 2*i + 2

  // nếu nút con trái lớn hơn so với gốc
  if (left < n && arr[left] > arr[largest])
    largest = left;

  // nếu nút con phải lớn hơn so với gốc
  if (right < n && arr[right] > arr[largest])
    largest = right;

  // nếu gốc không phải là lớn nhất
  if (largest != i){
    swap(arr[i], arr[largest]);

    // đệ quy lại hàm heapify()
    heapify(arr, n, largest);
  }
}

// Hàm vun đống
void heapSort(int arr[], int n){
  // tạo một đống (sắp xếp lại mảng)
  for (int i = n / 2 - 1; i >= 0; i--)
    heapify(arr, n, i);

  // trích xuất từng phần tử một từ heap
  for (int i = n - 1; i >= 0; i--){
    // di chuyển gốc về cuối
    swap(arr[0], arr[i]);

    // gọi hàm heapify
    heapify(arr, i, 0);
```

```
        }
    }

}
```

9.3 NỘI DUNG THỰC HÀNH

9.3.1 Câu 1:

Cho đoạn mã sau:

- File **thuvien_lab9.h** như sau:

Dòng	Mã lệnh
1	`#include<iostream>`
2	`using namespace std;`
3	`const int MAX = 50;`
4	`//Cấu trúc dãy số`
5	`struct NumberSeries{`
6	`    int a[MAX];`
7	`    int n;          //Số phần tử trong dãy số`
8	`};`
9	`void init(NumberSeries &series);`
10	`int getLength(NumberSeries series);`
11	`void createSeries(NumberSeries &series);`
12	`void printSeries(NumberSeries series);`

- File **caidat_lab9.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab9.h"`
2	`//Hàm khoi tao`
3	`void init(NumberSeries &series){`
4	`    series.n = 0;`
5	`}`
6	`int getLength(NumberSeries series){`
7	`    return series.n;`
8	`}`
9	
10	`void createSeries(NumberSeries &series){`
11	`    cout << "\n enter number of series:";`
12	`    cin >> series.n;`
13	`    for (int i = 0; i<series.n; i++){`
14	`        cout << "\n enter item " << i + 1 << ":";`
15	`        cin >> series.a[i];`
16	`    }`

```
}
void xuatDanhSach(NumberSeries series){
      cout << "\n Items in series:";
      for (int i = 0; i<series.n; i++)
            cout << series.a[i] << " ";
}
```

Yêu cầu:

Viết vào file **chuongtrinh_lab9.cpp** để gọi và chạy các hàm đã viết.

9.3.2 Câu 2:

Sau đây là hàm sắp xếp **selectionSort** để sắp xếp một **dãy số** tăng dần.

Dòng	Mã lệnh
1	`void selectionSort(NumberSeries &series){`
2	`  int  temp;`
3	`  int min;`
4	`  for (int i = 0; i<series.n - 1; i++){`
5	`  min = i;`
6	`  for (int j = i + 1; j<series.n; j++)`
7	`  if (series.a[j] <series.a[min])`
8	`  min = j;`
9	`  temp = series.a[i];`
10	`  series.a[i] = series.a[min];`
11	`  series.a[min] = temp;`
12	`  }`
13	`}`

Yêu cầu:

a/. Viết hàm **selectionSort** vào chương trình và viết void main gọi hàm trên

b/. Viết hàm **insertionSort** để sắp xếp dãy số giảm dần.

- ở file **chuongtrinh_lab9.cpp** như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab9.h"`
2	`void main(){`
3	`  NumberSeries series;`
4	
5	`  init(series);`
6	`  createSeries(series);`
7	`  printSeries(series);`
8	`  cout << "\n after selection sort:";`

```
9       selectionSort(series);
10      printSeries(series);
11  }
12
```

9.3.3 Câu 3:

Cho đoạn mã Quick sort sau, sinh viên viết lại cho phù hợp với cấu trúc dãy số và gọi hàm trong void main ().

Cài đặt:

```
void quickSort(int a[], int left, int right)
{       int     i, j, x;
        x = a[(left+right)/2];
        i = left; j = right;
        while(i < j)
        {
                while(a[i] < x) i++;
                while(a[j] > x) j--;
                if(i <= j)
                {
                        swap(a[i],a[j]);
                        i++ ; j--;
                }
        }
        if(left<j)
                quickSort(a, left, j);
        if(i<right)
                quickSort(a, i, right);
}
```

Hàm main gọi thực thi hàm Quick sort như sau:

Dòng	Mã lệnh
1	`#include"thuvien_lab9.h"`
2	`void main(){`
3	`NumberSeries series;`
4	
5	`init(series);`
6	`createSeries(series);`
7	`printSeries(series);`

```
8       cout << "\n After Quick sort:";
9       quickSort(series);
10       printSeries(series);
11
12  }
13
```

9.3.4 Câu 4:

Cho đoạn mã Heap sort như trong phần lý thuyết, sinh viên viết lại cho phù hợp với cấu trúc dãy số và gọi hàm trong void main ().

9.4 BÀI TẬP VỀ NHÀ

9.4.1 Câu 1:

Tạo cấu trúc môn học và danh sách môn học như trong bài lab 2. Sau đó viết hàm sắp xếp danh sách môn học tăng dần theo số tín chỉ (áp dụng giải thuật Selection sort, insertion sort, bubble sort)

9.4.2 Câu 2:

Áp dụng giải thuật Quick sort để sắp xếp danh sách môn học giảm dần theo mã môn học.

10 THI CUỐI KỲ